Smart AI Micro-Internship System

Muhammad Osama Ahmad(2280159)
Muhammad Siyaf (2280161)
Fahad Maqbool (2280142)

Supervised by: Mr. Muhammad Qasim

In partial fulfillment of requirement for the degree
Bachelor of Science (Software Engineering)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
Islamabad, Pakistan

Spring, 2026

Smart AI Micro Internship System

By

M. Osama Ahmad ***Fahad Maqbool*** ***M. Siyaf***

CERTIFICATE

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING (BSSE).

It is to certify that the above students thesis has been completed to my satisfaction and, to my belief, its standard is appropriate for submission for Evaluation. I have also conducted plagiarism test of this thesis using HEC prescribed software and found the similarity index to be within the permissible limit set by the HEC for the BSSE degree thesis. I have also found the thesis in a format recognized by the SZABIST for the BSSE thesis.

(Supervisor)	(FYP Committee Head)
Mr. Muhammad Qasim	Name

(BSSE Program Manager)
Sheikh Abdul Wahab

Date: _____

Department of Computer Science, Shaheed Zulfikar Ali Bhutto Institute of Science and Technology, Islamabad Campus

Spring, 2026

Project Overview

The Smart AI Micro-Internship Program is designed to address the gap between academic learning and industry requirements. Many students graduate with theoretical knowledge but lack practical experience, while organizations face difficulty in finding candidates with the right skills. This creates a mismatch between education and employment, making it important to develop a system that provides real-world learning opportunities.

The main challenge in solving this problem is to create a platform that not only connects students and employers but also evaluates skills in a structured and efficient way. Existing systems such as freelance platforms and job portals mainly focus on hiring and do not provide proper learning, performance tracking, or skill validation mechanisms.

To overcome these challenges, the proposed system provides a web-based platform that offers structured micro-internships. It allows students to complete real-world tasks, receive feedback, and improve their skills. The system uses intelligent matching techniques to recommend tasks based on user profiles and performance. It also includes features such as automated evaluation, progress tracking, and digital certification.

The approach followed in this project is based on an incremental development model, where the system is developed in phases including requirement gathering, system design, development, testing, and deployment. The scope of the project includes modules such as user management, task management, AI-based recommendation, performance evaluation, and analytics dashboard.

Compared to existing systems, the proposed solution provides a more complete and structured environment for both learning and recruitment. It focuses on skill-based evaluation rather than traditional resume-based hiring, making it more effective and reliable.

In conclusion, the Smart AI Micro-Internship Program provides an innovative solution for improving employability and recruitment processes. In the future, the system can be enhanced by adding more advanced AI models, expanding to multiple domains, and integrating with educational platforms to further improve learning and career development opportunities.

Dedication

Firstly, we dedicate our project to the creator Allah Almighty and dedicate to whom the world owes its existence Muhammad (Peace Be Upon Him) and dedicate this to our beloved parents, our extremely dedicated and generous teachers and supportive friends, their prayers always pave the way to success for us.

Acknowledgement

First and foremost, we are deeply grateful to Allah Almighty, whose blessings, guidance, and mercy gave us the strength, patience, and determination to complete this project successfully. Without His help, none of our efforts would have reached a meaningful conclusion.

We would like to express our sincere gratitude to our supervisor, Mr. Muhammad Qasim, for his continuous guidance, valuable feedback, and constant encouragement throughout every stage of this project. His insightful suggestions and constructive criticism played a vital role in shaping the direction of our work and improving the overall quality of the system. We are truly thankful for the time and effort he devoted to mentoring us.

We also extend our heartfelt thanks to the faculty members and staff of the Department of Computer Science at the Shaheed Zulfikar Ali Bhutto Institute of Science and Technology (SZABIST), Islamabad Campus, for providing us with a supportive learning environment and the resources necessary to carry out this project. The knowledge and skills we gained during our degree formed the foundation upon which this work was built.

Finally, we are profoundly grateful to our beloved parents and family members for their endless love, prayers, and financial and emotional support, and to our friends and classmates for their cooperation and motivation during difficult times. Their unwavering belief in us was a constant source of inspiration throughout this journey.

(Student 1)
M. Osama Ahmad

(Student 2)
Fahad Maqbool

(Student 3)
M. Siyaf

Revision History

The following table presents the revision history of this document. Each entry records the date of revision, the team member(s) responsible for compiling or updating the content, the reason for the revision, and the corresponding version number. This revision history ensures traceability and accountability throughout the documentation process of the Smart AI Micro-Internship Program.

Date	**Compiled By**	**Reason for Revision**	**Version**
Feb 15, 2026	M. Osama Ahmad, Fahad Maqbool, M. Siyaf	Initial draft of the project proposal and introduction chapter	1.0
Mar 01, 2026	M. Osama Ahmad	Completion of requirement gathering phase and SRS document	1.1
Mar 15, 2026	Fahad Maqbool	Addition of functional and non-functional requirements	1.2
Apr 01, 2026	M. Siyaf	System design chapter including use case and class diagrams	1.3
Apr 10, 2026	M. Osama Ahmad, Fahad Maqbool	Software project plan, scheduling, and risk management sections	1.4
Apr 14, 2026	M. Osama Ahmad, Fahad Maqbool, M. Siyaf	Review and revision of all chapters, formatting corrections, and reference updates	2.0

Table 1: Document Revision History

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Universities across the region produce hundreds of thousands of graduates every year, yet a persistent complaint from both students and employers remains the same: graduates arrive with solid theoretical knowledge but little hands-on experience, while organisations struggle to find task-ready candidates for short, well-defined pieces of work. This mismatch is the core problem our team set out to solve. The Smart AI Micro-Internship Program is an intelligent web-based platform that connects students with organisations through AI-matched, short-term task engagements, turning academic capability into demonstrable, verified work output.

Unlike traditional internships that require months of commitment, our platform structures the experience around focused micro-tasks. A machine-learning matching engine pairs each student with opportunities that fit their actual skill profile rather than relying on keyword searches. The full lifecycle — from application and mentor-supervised task completion through to automated evaluation and verified certificate issuance — happens inside a single platform. For organisations, especially startups and SMEs, this removes the usual friction of hiring: instead of sifting through static resumes, they receive a ranked list of candidates whose compatibility with the task's requirements has already been computed.

This document records the complete engineering process behind the platform. Chapter 2 specifies the functional and non-functional requirements gathered from potential users. Chapter 3 covers project planning, scheduling, and risk management. Chapter 4 presents the functional analysis through use cases, entity models, and data-flow diagrams. Chapter 5 walks through the system design using UML class, deployment, activity, and sequence diagrams. The subsequent chapters address the interface and physical design, testing strategy, and the user manual. Each chapter builds directly on the one before it, forming a coherent engineering specification from problem definition to working solution.

1.1 Product Purpose

The **Smart AI Micro-Internship Program** is designed to address the growing gap between academic learning and industry requirements by providing a structured platform for skill-based micro-internships. In the current environment, students often graduate with strong theoretical knowledge but lack practical exposure, while organizations face challenges in identifying suitable candidates for short-term, skill-specific tasks. This mismatch highlights the need for a system that enables practical experience acquisition and efficient talent evaluation [1].

The project aims to provide a comprehensive solution by developing an intelligent web-based platform that connects students with industry tasks through AI-driven matching and evaluation. The system follows a structured workflow where students create profiles, receive task recommendations based on their skills, complete assigned tasks, and are evaluated through automated analytics. Employers can post tasks, review performance

metrics, and identify suitable candidates based on real work outcomes rather than static resumes, ensuring a more accurate and efficient matching process.

The development of the Smart AI Micro-Internship Program is carried out using modern web technologies and machine learning techniques to ensure scalability, efficiency, and adaptability. The project progresses through clearly defined phases, including requirement analysis, system design, development of front-end and back-end modules, integration of AI components, and testing. Major milestones include completion of system architecture, deployment of core modules, integration of intelligent matching algorithms, and final system deployment within the planned timeline.

The expected result of this project is a fully functional web application that enhances employability by providing verified, real-world experience to students while enabling organizations to efficiently source talent. The project is expected to be completed by December 2026, with success measured through system usability, accuracy of task recommendations, and user satisfaction. Overall, the Smart AI Micro-Internship Program is anticipated to improve the current recruitment and learning ecosystem by introducing a data-driven, performance-based approach to skill validation and talent acquisition.

1.2 Project Scope and Module Breakdown

The scope of the Smart AI Micro-Internship System encompasses the complete development and deployment of an intelligent web-based micro-internship platform designed to bridge the gap between academic education and industry requirements. The system aims to provide a structured digital environment where students can develop practical skills through short, task-based internships while organizations can efficiently discover and evaluate potential talent. Overall, the Smart AI Micro-Internship System aims to create a structured ecosystem where students gain practical exposure, organizations access skilled candidates, and mentors facilitate professional development. By integrating intelligent matching, automated evaluation, and performance analytics, the platform promotes a more efficient, transparent, and scalable approach to experiential learning [2].

The primary modules within the project scope are as follows:

1.2.0.1 Module 1: Student Profile Management The Student Profile Management module allows students to create and maintain detailed profiles within the platform. These profiles include academic background, technical skills, interests, and previous task experiences. By maintaining structured student profiles, the system can accurately represent each user's capabilities and enable better interaction between students and employers.

From our team's perspective, this module is essential because it forms the foundation for the intelligent matching functionality of the platform. A well-structured student profile allows the system to analyze skills effectively and recommend appropriate micro-internship opportunities that align with each student's abilities and interests.

1.2.0.2 Module 2: Company Profile Management The Company Profile Management module enables organizations to register and create professional profiles within the system. Companies can provide information about their industry domain, organizational details, and internship opportunities, allowing students to better understand potential

employers before applying.

From our team's perspective, this module improves transparency and credibility within the platform. By allowing companies to maintain verified profiles, the system ensures that students interact with legitimate organizations while also providing employers with a centralized environment to manage their internship activities.

1.2.0.3 Module 3: Task / Micro-Internship Posting The Task or Micro-Internship Posting module allows organizations to publish short-term tasks that students can apply for. Employers can define task descriptions, required skills, expected deliverables, and deadlines. This structured task posting process ensures that students clearly understand the expectations associated with each opportunity.

From our team's perspective, this module is a core component of the platform because it connects industry needs with student capabilities. By allowing organizations to easily post real-world tasks, the system encourages active participation from employers while giving students meaningful opportunities to gain practical experience.

1.2.0.4 Module 4: Application Tracking The Application Tracking module enables students and organizations to monitor the status of internship applications throughout the selection process. Students can track whether their applications are under review, accepted, or rejected, while employers can manage and review submitted applications efficiently.

From our team's perspective, this module enhances transparency and communication between both parties. Many existing platforms lack clear application tracking mechanisms, which can create confusion for applicants. Our system addresses this issue by providing clear status updates and streamlined application management.

1.2.0.5 Module 5: Interview Scheduling The Interview Scheduling module provides tools that allow organizations to arrange interviews with shortlisted candidates. Employers can schedule interview sessions directly within the platform, and students receive notifications regarding the scheduled time and meeting details.

From our team's perspective, integrating interview scheduling into the platform simplifies the recruitment workflow and reduces dependency on external communication channels. This feature helps both students and employers coordinate efficiently while maintaining a structured recruitment process.

1.2.0.6 Module 6: AI-Based Matching The AI-Based Matching module uses intelligent algorithms to analyze student profiles and match them with suitable tasks. The system evaluates skills, interests, and previous performance to recommend the most relevant opportunities to students and appropriate candidates to employers.

From our team's perspective, this module represents one of the most innovative aspects of the system. By automating the matching process, the platform reduces manual searching and increases the likelihood of successful task completion through better compatibility between students and internship requirements.

1.2.0.7 Module 7: Mentor Assignment The Mentor Assignment module allows experienced professionals or supervisors to guide students throughout the completion of

micro-internship tasks. Mentors can monitor progress, provide advice, and assist students in solving challenges during task execution.

From our team's perspective, mentorship significantly improves the learning experience for students. Instead of completing tasks independently without guidance, students receive professional support that enhances their understanding of real-world industry practices.

1.2.0.8 Module 8: Progress Tracking The Progress Tracking module enables continuous monitoring of task completion and student development during micro-internships. Students can update task milestones, while mentors and organizations can review progress reports and performance indicators.

From our team's perspective, progress tracking ensures accountability and structured learning. This feature helps maintain project timelines while allowing mentors to intervene when additional support is required.

1.2.0.9 Module 9: Automated Evaluation The Automated Evaluation module assesses student performance after task completion based on predefined evaluation criteria. Mentors or employers can review submissions, assign ratings, and record evaluation outcomes within the system.

From our team's perspective, automated evaluation improves fairness and consistency in performance assessment. The system ensures that evaluations are structured and properly documented, enabling reliable measurement of student performance.

1.2.0.10 Module 10: Feedback System The Feedback System allows mentors and employers to provide detailed comments and suggestions to students after completing tasks. Constructive feedback helps students understand their strengths and areas requiring improvement.

From our team's perspective, feedback is a critical component of professional growth. By incorporating a dedicated feedback mechanism, the system encourages continuous learning and skill enhancement for students.

1.2.0.11 Module 11: Performance Analytics Dashboard The Performance Analytics Dashboard provides visual insights into student activity, skill development, and task completion rates. The system presents analytical reports that help users understand performance trends and productivity levels.

From our team's perspective, this module enables data-driven decision-making for both students and organizations. Students can monitor their professional growth, while organizations can identify high-performing candidates for future opportunities.

1.2.0.12 Module 12: Payment Integration The Payment Integration module enables secure financial transactions for paid micro-internship tasks. Organizations can provide compensation to students through integrated digital payment systems.

From our team's perspective, integrating payments encourages greater participation from both students and organizations. This feature supports fair compensation while maintaining transparency in financial transactions.

1.2.0.13 Module 13: Document Verification The Document Verification module allows the system to verify important documents such as student credentials or identity information. Verification helps ensure the authenticity of participants within the platform.

From our team's perspective, this module strengthens the credibility and reliability of the system. By preventing fraudulent profiles, the platform maintains a secure and trustworthy environment for both students and organizations.

Serial #	Abbreviation	Definition
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	API	Application Programming Interface
4	UI	User Interface
5	UX	User Experience
6	DB	Database
7	REST	Representational State Transfer
8	JWT	JSON Web Token
9	IDE	Integrated Development Environment
10	QA	Quality Assurance
11	SME	Small and Medium Enterprise
12	NLP	Natural Language Processing
13	KPI	Key Performance Indicator
14	SDLC	Software Development Life Cycle
15	API	Application Programming Interface
16	UI/UX	User Interface and User Experience
17	NoSQL	Non-Relational Database
18	SQL	Structured Query Language
19	CI/CD	Continuous Integration / Continuous Deployment
20	OAuth	Open Authorization Protocol

Table 1.1: Abbreviations

1.2.1 Existing System Description

In recent years, numerous web-based platforms have attempted to reduce the gap between students and industry by providing freelance opportunities, internships, and skill-based recruitment systems. These platforms aim to connect individuals with organizations seeking talent; however, most of them function primarily as job marketplaces rather than structured learning environments. While they allow employers to post projects and candidates to submit applications, many systems lack mechanisms for intelligent performance evaluation, verified skill validation, and structured career development support. As a result, students often struggle to demonstrate practical competencies beyond traditional resumes. This section reviews several existing platforms and evaluates their strengths and limitations in comparison with the proposed Smart AI Micro-Internship

System.

In addition to freelance and internship portals, several learning management and project collaboration platforms have attempted to improve practical exposure through online learning environments. Platforms such as Coursera and GitHub Classroom provide structured courses, coding assignments, and project submissions that help students develop technical skills. However, these systems primarily focus on education delivery and collaborative project management rather than direct engagement with industry professionals. Although students can improve their skills through these platforms, they often lack mechanisms that connect learners directly with organizations offering real-world tasks or internships [3, 4]. Consequently, the transition from academic learning to verified professional experience remains limited.

Furthermore, modern recruitment technologies increasingly incorporate artificial intelligence to automate candidate screening and shortlisting processes. Many AI-based hiring systems utilize resume parsing, keyword extraction, and automated ranking algorithms to evaluate applicants. While these methods can improve hiring efficiency, they frequently rely on static resume data rather than assessing real task performance or project outcomes [5]. This limitation can result in inaccurate assessments of a candidate's practical abilities, especially for students who may have limited professional experience but possess strong technical skills.

Recent research on intelligent job matching systems highlights the potential of machine learning techniques for skill extraction, candidate ranking, and predictive analytics in recruitment systems [2]. These approaches demonstrate that combining AI-based skill profiling with real task performance data can significantly improve hiring accuracy and reduce bias in recruitment processes. However, most existing implementations are designed for large-scale enterprise hiring rather than student-oriented micro-internship ecosystems.

Therefore, despite the availability of various freelance platforms, internship portals, and AI-driven recruitment systems, there remains a lack of integrated platforms that combine structured micro-internships, intelligent skill matching, automated evaluation, and verified certification within a single system. The proposed Smart AI Micro-Internship System aims to address these limitations by integrating intelligent task matching, performance-based evaluation, and verifiable certification within a scalable web-based platform designed specifically to support the transition from academic learning to industry engagement.

Upwork [6] is a globally recognized freelance platform that connects clients with independent professionals across a wide range of industries. The platform allows organizations to post projects while freelancers submit proposals based on their skills and experience. Upwork provides features such as milestone-based payments, secure communication channels, and freelancer rating systems that help build credibility and trust between clients and workers. Although the platform enables global collaboration and remote work opportunities, it primarily targets experienced freelancers rather than students seeking structured learning experiences.

The core workflow of Upwork is based on a competitive bidding model in which clients publish job posts and freelancers spend connects, a form of paid virtual currency,

to submit proposals. The platform supports both fixed-price and hourly contracts, and it includes a built-in time-tracking tool, an escrow-based payment protection system, and a dispute resolution service. Reputation on Upwork is accumulated through a Job Success Score and client reviews, which strongly influence how often a freelancer is shortlisted. This reputation-driven design rewards professionals who already have a long history of completed contracts, which creates a significant entry barrier for newcomers.

From the perspective of students and the proposed system, Upwork is poorly suited to structured skill development. New users with little or no track record struggle to win their first contracts because clients consistently favour highly rated freelancers, leaving beginners trapped in a cycle where they cannot gain experience without first having experience. The platform offers no learning pathway, no mentorship, and no automated feedback to help a beginner improve, and its matching is driven by manual client selection rather than by intelligent analysis of a candidate's skills. In contrast, the Smart AI Micro-Internship Program is designed specifically around skill-based learning, AI-driven matching, and structured evaluation, addressing exactly the gaps that make Upwork unsuitable for students entering the workforce.

Features:

- Project posting and bidding system
- Freelancer profile with rating and review system
- Secure payment and milestone tracking
- Messaging and collaboration tools
- Global client and freelancer access

The limitations of the application are listed below.

Limitations:

- No AI-based skill gap analysis
- Limited career guidance support
- High competition for beginners
- No structured micro-internship framework
- Manual evaluation of submissions

Internshala [**7**] is an internship-focused platform designed to connect students with organizations offering internship opportunities. It provides categorized listings of internships, resume submission features, and additional skill development courses aimed at improving student employability. While the platform simplifies internship discovery and application processes, it mainly operates as a listing service and does not provide advanced intelligent matching or automated evaluation mechanisms.

Internshala combines an internship marketplace with a set of paid training courses covering areas such as web development, digital marketing, and data science. Students browse listings using filters for category, location, stipend, and duration, and then apply by submitting a profile and a short cover note. The platform also issues completion certificates for its own training programs and offers a placement guarantee track for some

courses. Its strength lies in being student-oriented and accessible, with a large volume of entry-level opportunities that are easy to discover and apply to.

Despite these strengths, Internshala functions primarily as a directory rather than an intelligent, performance-driven system. The matching between students and internships is based on simple keyword filters and manual employer screening, so students frequently receive listings that are only loosely related to their actual skills. The platform does not evaluate the quality of a student's work on a task, does not generate data-driven feedback, and does not track measurable performance across internships, which means a student's profile remains a static résumé rather than a verified record of demonstrated ability. The proposed Smart AI Micro-Internship Program improves on this model by replacing keyword filtering with AI-based skill matching, and by adding automated evaluation, performance analytics, and verifiable certificates earned through real task completion rather than through paid courses alone.

Features:

- Internship search and filtering options
- Student profile and resume submission
- Online skill development courses
- Application tracking system
- Employer and student communication tools

The limitations of the application are listed below.

Limitations:

- No automated AI-based candidate matching
- Limited real-time performance analytics
- Lack of predictive career path modeling
- No integrated intelligent evaluation system
- Internship duration often long and rigid

LinkedIn Jobs [8] is part of the LinkedIn professional networking ecosystem and serves as a global recruitment platform where companies advertise job opportunities and professionals apply directly. The platform provides features such as professional networking, skill endorsements, company pages, and applicant tracking tools. Although LinkedIn is highly effective for professional hiring and networking, it primarily focuses on full-time employment opportunities rather than short-term skill-based micro-internships for students.

LinkedIn integrates job discovery with a rich professional identity, allowing users to build a profile, connect with industry professionals, follow companies, and obtain endorsements and recommendations that signal credibility. Its recruitment tools include applicant tracking for employers, a recommendation engine that suggests relevant jobs, and analytics that show how a candidate compares with other applicants. These features make LinkedIn a powerful tool for established professionals and for organizations seeking to fill permanent positions, and its skill-assessment quizzes provide a limited form of competency signalling.

However, for the specific goal of structured micro-internships, LinkedIn presents several limitations. Its opportunities are overwhelmingly oriented toward full-time and senior roles, and even its job recommendations rely heavily on declared profile keywords and network connections rather than on verified, task-based performance. The platform provides no mechanism for assigning a short structured task, evaluating the submitted work automatically, or tracking a student's measurable progress over time, so a student with strong practical ability but a thin network is easily overlooked. The proposed system differs fundamentally by focusing on small, well-defined tasks with clear deliverables, by matching students through an AI engine that analyses concrete skills and project history, and by validating competence through completed work rather than through endorsements and connections.

Features:

- Professional networking and job postings
- Skill endorsements and recommendations
- Company profiles and job insights
- Applicant tracking tools
- Global employment marketplace

The limitations of the application are listed below.

Limitations:

- No micro-task structured internship model
- Limited AI-driven task recommendation
- No automated submission evaluation
- No structured student performance dashboard
- Focused mainly on full-time employment

Table 1.2 presents a comparative analysis of the reviewed applications. The comparison focuses on key parameters such as intelligent task matching, structured micro-internship models, automated evaluation mechanisms, performance analytics capabilities, and verified certificate generation.

- AI-Based Matching (intelligent task recommendation)
- Structured Micro-Internship Model
- Automated Evaluation Support
- Performance Analytics Dashboard
- Verified Certificate Generation

Fiverr [**9**] is a popular freelance marketplace that enables individuals to offer services (known as gigs) across various categories such as programming, design, and digital marketing. It allows freelancers to showcase their skills through predefined service packages, while clients can directly purchase services without a bidding process. Fiverr simplifies

service delivery and provides a global platform for freelancers; however, it is primarily designed for commercial services rather than structured learning or internship-based experiences.

Unlike bidding-based marketplaces, Fiverr uses a catalogue model in which freelancers publish fixed-price gigs with tiered packages, and buyers purchase them directly much like products in an online store. The platform handles payments, order tracking, and delivery deadlines, and it ranks gigs through an internal algorithm that rewards sellers with strong ratings, fast response times, and consistent on-time delivery. This model is efficient for buyers who need a specific, well-defined service and allows skilled sellers to productize their expertise without negotiating each contract individually.

For students seeking to build verified, industry-relevant experience, however, Fiverr offers little structure or guidance. Success on the platform depends on marketing one's own gigs and competing on price and ratings, which disadvantages beginners who have no reviews and no portfolio, and there is no mentorship or learning pathway to help them improve. Crucially, Fiverr evaluates outcomes only through buyer star ratings rather than through any structured, criteria-based assessment of skill, and it issues no verifiable academic or professional certification. The Smart AI Micro-Internship Program addresses these shortcomings by providing structured tasks with defined deliverables, mentor support, automated and criteria-based evaluation, and verified certificates, thereby converting practical work into recognised proof of competence rather than an isolated commercial transaction.

Features:

- Gig-based service marketplace
- Freelancer portfolio and rating system
- Secure payment and order tracking
- Service categorization and search filters
- Direct client–freelancer interaction

The limitations of the application are listed below.

Limitations:

- No structured internship framework
- Lack of AI-based skill matching
- No performance-based evaluation system
- Limited focus on student learning and development
- No verified certification mechanism

Handshake [**10**] is a career development platform designed specifically for students and recent graduates. It connects universities with employers and provides internship and job opportunities tailored to students. The platform includes features such as employer connections, career fairs, and application tracking. While Handshake is effective in linking students with employers, it mainly focuses on job discovery rather than structured micro-internship experiences with automated evaluation.

Handshake is distinctive because it is integrated directly with universities, which verify student identities and academic records before granting access. This institutional integration gives employers confidence in the authenticity of applicants and allows the platform to host virtual career fairs, employer information sessions, and curated event recommendations. For students, Handshake centralises the early-career job search and provides a trusted environment in which to discover internships and graduate roles that are specifically targeted at their level of experience.

Nevertheless, Handshake remains fundamentally a discovery and networking platform rather than a system for developing and verifying skills through real work. It matches students to opportunities using profile attributes, declared interests, and employer preferences, but it does not assign structured micro-tasks, does not assess the quality of completed work, and provides no automated, data-driven feedback or performance analytics. As a result, a student's standing on Handshake still depends largely on a conventional résumé rather than on demonstrated, measurable ability. The proposed Smart AI Micro-Internship Program builds on the student-centred philosophy of Handshake but extends it significantly by adding AI-based task matching, a structured micro-internship workflow, automated evaluation, performance dashboards, and verifiable certificates that together turn participation into concrete evidence of skill.

Features:

- Student-focused job and internship listings
- University-integrated platform access
- Employer networking and career events
- Application tracking and profile management
- Career guidance resources

The limitations of the application are listed below.

Limitations:

- No AI-driven task recommendation system
- Limited real-time performance tracking
- No structured micro-internship model
- Lack of automated evaluation mechanisms
- No integrated skill verification system

1.2.2 Future System Usage Analysis

The Smart AI Micro-Internship Program is designed with scalability and adaptability to support future expansion across multiple domains and user groups. As the system evolves, it is expected to accommodate a larger number of students, organizations, and institutions, enabling a broader ecosystem for skill-based learning and recruitment. The increasing demand for remote work and digital skill validation further strengthens the relevance of such platforms in modern workforce development [1].

In future deployments, the system can be extended to integrate with university portals and learning management systems, allowing seamless tracking of student performance and

Table 1.2: Applications Comparison

Features	**Applications**					
	Upwork [6]	Internshala [7]	LinkedIn [8]	Fiverr [9]	Handshake [10]	Proposed System
Company Profile Management	✓	✓	✓	✓	✓	✓
Application Tracking	✓	✓	✓	✓	✓	✓
Interview Scheduling	✓	✓	✓	✗	✓	✓
Mentor Assignment	✗	✗	✗	✗	✗	✓
Progress Tracking	✗	✗	✗	✗	✗	✓
Feedback System	✓	✓	✓	✓	✓	✓
Payment Integration	✓	✓	✗	✓	✗	✓
Document Verification	✗	✗	✓	✗	✓	✓
AI-Based Matching	✗	✗	✗	✗	✗	✓
Structured Micro-Internship Model	✗	✗	✗	✗	✗	✓
Automated Evaluation	✗	✗	✗	✗	✗	✓
Performance Analytics Dashboard	✗	✗	✗	✗	✗	✓
Verified Certificates	✗	✓	✗	✗	✗	✓

academic progress. Additionally, advanced AI models can be incorporated to enhance recommendation accuracy, including deep learning-based profiling and natural language processing techniques for analyzing resumes, project descriptions, and task requirements. This will improve the system's ability to provide personalized career pathways and predictive performance insights.

The platform also has the potential to support industry-specific modules, enabling customization for sectors such as software development, business analytics, healthcare, and engineering. Integration with external APIs and professional networking platforms can further enhance visibility and credibility of student achievements. Moreover, the system can evolve into a data-driven decision support tool for organizations by providing analytics on skill trends, workforce demand, and candidate performance patterns.

From an operational perspective, cloud-based deployment and containerization technologies will allow the system to scale efficiently with increasing user load. Continuous updates through CI/CD pipelines can ensure system reliability, security, and feature enhancement over time. Ultimately, the Smart AI Micro-Internship Program is expected to become a comprehensive digital ecosystem that supports lifelong learning, efficient talent acquisition, and data-driven workforce development, significantly improving upon current fragmented solutions.

1.3 Objectives

The primary objective of the Smart AI Micro-Internship Program is to provide an intelligent, scalable, and efficient platform that bridges the gap between academic learning and industry requirements. The system is designed to serve multiple user groups, each with specific needs and expectations, while ensuring seamless interaction between stakeholders and end-users.

1.3.1 Primary Objectives

The system aims to enhance employability by providing students with structured, short-term micro-internship opportunities that focus on real-world task execution. It also seeks to enable organizations to efficiently identify, evaluate, and recruit suitable candidates based on performance rather than traditional resume-based screening methods. By integrating artificial intelligence, the platform intends to deliver accurate skill matching, predictive recommendations, and automated performance evaluation [2].

1.3.2 Intended Users

The primary users of the system include university students and fresh graduates who seek practical experience and industry exposure. These users interact with the system by creating profiles, participating in micro-internships, and tracking their performance. Employers and organizations also serve as key users, utilizing the platform to post tasks, evaluate submissions, and identify potential candidates for recruitment.

1.3.3 Stakeholders

The stakeholders of the system include educational institutions, industry partners, and recruitment agencies that benefit indirectly from improved skill validation and talent acquisition processes. Universities can use the system to monitor student progress and enhance curriculum alignment with industry needs, while organizations can leverage analytics and performance data for informed hiring decisions.

1.3.4 System Objectives

From a technical perspective, the system aims to achieve modularity, scalability, and reliability through the use of modern web technologies and AI-driven components. It focuses on providing secure authentication, efficient data management, and real-time analytics to support decision-making processes. Additionally, the platform is designed to ensure user-friendly interaction, maintain data integrity, and support future expansion with minimal system modifications.

1.3.5 Expected Outcomes

The successful implementation of the Smart AI Micro-Internship Program is expected to result in improved employability for students, reduced recruitment costs for organizations, and enhanced collaboration between academia and industry. By addressing the limitations of existing platforms, the system aims to provide a more effective and data-driven solution for skill development and talent management.

1.4 Problem Statement

The rapid growth of higher education and digital learning platforms has increased the number of graduates entering the job market each year. However, a significant gap remains between academic learning and industry requirements. According to the World Economic Forum, over 50% of the global workforce requires reskilling due to evolving job demands and technological advancements [1]. Despite obtaining degrees, many students lack practical, project-based experience, which reduces their employability and competitiveness in the job market.

At the same time, companies—particularly startups and small-to-medium enterprises require skilled individuals for short-term or project-based tasks. Traditional hiring processes are time-consuming, expensive, and often inefficient for micro-level assignments. Reports indicate that recruitment costs can reach up to 20% of an employee's annual salary, making it impractical for short-duration tasks [11]. Additionally, many recruitment platforms rely heavily on resume-based screening rather than performance-based evaluation, limiting the accuracy of candidate selection.

Existing web-based platforms such as freelance marketplaces and internship portals provide partial solutions by connecting students and employers. However, these platforms primarily function as listing services and lack structured micro-internship frameworks, intelligent skill analysis, and automated evaluation mechanisms. As a result, both students and companies remain underserved. Students struggle to gain verified practical experience, while companies face challenges in identifying reliable and task-ready candidates efficiently.

The unmet need, therefore, lies in the absence of a unified web-based system that integrates short-term micro-internships, AI-based adaptive matching, performance prediction, automated evaluation, and verified certification within a single ecosystem. The significance of this problem is substantial, particularly in developing economies where youth unemployment rates remain high. The International Labour Organization reports that global youth unemployment exceeds 13%, highlighting the urgent need for scalable employment-support systems [12].

The primary customer segments for such a solution include:

- **University Students and Fresh Graduates:** Globally, more than 235 million students are enrolled in higher education institutions, representing a large potential user base seeking practical exposure [13].
- **Startups and SMEs:** Small and medium enterprises account for approximately 90% of businesses worldwide and often require cost-effective, flexible talent solutions [14].
- **Educational Institutions:** Universities aiming to improve graduate employability through structured industry collaboration.
- **Recruitment and HR Departments:** Organizations seeking data-driven hiring tools for short-term project evaluation.

From a research perspective, this problem addresses the need for integrating artificial intelligence into skill assessment and predictive recruitment systems. The proposed solution can be applied in academic institutions, corporate HR departments, online learning ecosystems, and digital workforce platforms. By combining AI-driven skill intelligence with real task execution tracking, the system aims to reduce hiring inefficiencies, enhance employability, and support data-driven workforce development.

Therefore, the proposed Smart AI Micro-Internship System system aims to solve a significant and measurable problem by creating a structured, intelligent, and scalable micro-internship ecosystem that bridges the persistent gap between education and industry requirements.

1.4.1 Significance of the Problem

The significance of this issue is measurable at both global and regional levels. Youth unemployment remains a pressing challenge worldwide. The International Labour Organization reports that global youth unemployment exceeds 13%, with even higher rates in developing economies [12]. This indicates that millions of graduates struggle to transition from education to employment due to limited practical exposure and insufficient industry alignment.

Furthermore, the global higher education population exceeds 235 million students, representing a vast demographic seeking experiential learning opportunities [13]. Simultaneously, small and medium enterprises account for approximately 90% of businesses worldwide and frequently require cost-effective and project-based talent solutions [14]. The absence of an intelligent micro-internship ecosystem therefore impacts both talent supply and industry demand at scale.

1.5 Proposed Solution

The proposed solution, titled **Smart AI Micro-Internship Program**, is an intelligent web-based platform designed to bridge the gap between academic learning and industry requirements. The system provides a structured environment where students can participate in short-term, skill-based micro-internships, while organizations can efficiently evaluate and recruit talent based on real task performance. By integrating artificial intelligence with modern web technologies, the platform ensures accurate skill matching, automated evaluation, and data-driven decision-making [2].

The system addresses the limitations of existing platforms by introducing a unified ecosystem that combines task-based learning, intelligent recommendation, and performance analytics. Unlike traditional job portals that rely on resumes, the proposed solution focuses on practical skill validation through real-world tasks. The system is designed to be scalable, user-friendly, and adaptable to multiple domains, ensuring long-term usability and expansion.

1.5.1 System Modules

The Smart AI Micro-Internship Program is divided into several key modules, each responsible for specific functionalities within the system.

1.5.1.1 Administrator Module The Administrator module is responsible for overall system management, monitoring, and control. It ensures that all system operations are functioning correctly and that users and data are managed efficiently.

- Manage user profiles, including students and employers.
- Monitor system activities and performance analytics.
- Manage platform content, tasks, and system configurations.
- Handle user verification and access control.

1.5.1.2 Student Module The Student module enables users to interact with the system by creating profiles, participating in micro-internships, and tracking their progress. It focuses on providing a personalized learning and career development experience.

- Create and manage personal profiles with skills and qualifications.
- Receive AI-based task recommendations based on skill analysis.
- Submit completed tasks and track performance metrics.
- View certificates and achievement records.

1.5.1.3 Employer Module The Employer module allows organizations to post tasks, evaluate submissions, and identify suitable candidates based on performance data.

- Post micro-internship tasks with defined requirements.
- Review and evaluate student submissions.
- Access performance analytics and candidate insights.
- Select and recruit candidates based on task outcomes.

1.5.1.4 AI Recommendation and Evaluation Module This module forms the core intelligence of the system, enabling automated decision-making and personalized recommendations.

- Analyze student skills and match them with relevant tasks.
- Predict performance based on historical data and patterns.
- Provide automated evaluation and scoring mechanisms.
- Generate insights and recommendations for users and organizations.

1.5.2 Project Plan and Implementation Overview

The development of the system follows a structured approach that includes requirement analysis, system design, implementation, testing, and deployment. The platform will be developed as a web application using modern front-end and back-end technologies, integrated with machine learning models for intelligent functionality. The implementation process includes iterative development and continuous testing to ensure reliability and performance.

The project is planned to be completed within the defined timeline from February 2026 to December 2026, with key milestones including system design completion, module development, integration, testing, and final deployment. The system will be deployed on a cloud-based platform to ensure scalability and accessibility.

1.5.3 Expected Benefits

The proposed solution offers significant benefits to all stakeholders by improving employability, enhancing recruitment efficiency, and enabling data-driven decision-making. Students gain practical experience and verified skill validation, while organizations benefit from reduced hiring costs and improved candidate selection accuracy. Educational institutions can leverage the system to strengthen industry collaboration and enhance curriculum relevance.

Overall, the Smart AI Micro-Internship Program provides a comprehensive, scalable, and intelligent solution that improves upon existing systems by integrating structured micro-internships with AI-driven evaluation and recommendation capabilities.

1.6 Intended Market of Product

The Smart AI Micro-Internship Program is designed to serve a broad and rapidly growing market that includes students, educational institutions, and organizations seeking efficient talent acquisition solutions. The increasing demand for practical skill development and industry-ready graduates has created a significant opportunity for platforms that facilitate real-world experience and performance-based evaluation [1].

1.6.1 Target Market Segments

The primary market for the system consists of university students and fresh graduates who require structured opportunities to gain practical experience and enhance their employability. This segment represents a large and continuously expanding user base due to the global rise in higher education enrollment.

Another key segment includes startups and small-to-medium enterprises that require flexible, cost-effective talent solutions for short-term or project-based tasks. These organizations often face challenges in traditional hiring processes and can benefit significantly from a performance-based recruitment platform.

Educational institutions and training organizations also form an important part of the market, as they aim to improve student outcomes and align academic learning with industry requirements. Additionally, recruitment agencies and HR departments can utilize the system for data-driven candidate evaluation and hiring decisions.

1.6.2 Market Value and Worth of Product

The value of the Smart AI Micro-Internship Program lies in its ability to address inefficiencies in both skill development and recruitment processes. By integrating artificial intelligence with structured micro-internships, the system provides a unique solution that enhances employability while reducing recruitment costs.

From a market perspective, the platform has strong growth potential due to increasing digital transformation, remote work trends, and the demand for skill-based hiring models. The system can be scaled across different industries and geographic regions, making it a sustainable and adaptable solution.

1.6.3 Competitive Advantage

The proposed system differentiates itself from existing platforms through its integrated approach that combines AI-based recommendation, automated evaluation, and structured micro-internships within a single ecosystem. Unlike traditional job portals and freelance marketplaces, the platform focuses on practical skill validation and real task performance.

This competitive advantage positions the Smart AI Micro-Internship Program as a valuable solution in the evolving digital workforce landscape, offering benefits to both talent providers and employers while contributing to a more efficient and transparent recruitment process.

1.7 Intended Users of Product

The Smart AI Micro-Internship Program is designed to serve a diverse group of users who directly interact with the system for skill development, recruitment, and performance evaluation. These users represent the core audience of the platform and are essential for its effective operation and success.

1.7.1 Primary Users

The primary users of the system include students and fresh graduates who seek practical experience and industry exposure. These users utilize the platform to create profiles, participate in micro-internships, and track their performance through structured tasks and analytics. The system enables them to develop relevant skills and enhance their employability in a competitive job market [2].

Employers and organizations also form a key group of primary users. They interact with the platform by posting micro-internship tasks, evaluating student submissions, and identifying suitable candidates based on performance metrics. This allows organizations to make informed hiring decisions and reduce dependency on traditional recruitment methods.

1.7.2 Secondary Users

Secondary users include educational institutions such as universities and training centers. These institutions can use the platform to monitor student progress, evaluate outcomes, and align academic programs with industry requirements. The system also supports faculty members in guiding students toward practical skill development.

Recruitment agencies and HR professionals are another category of secondary users. They can leverage the system's analytics and performance data to improve candidate selection processes and enhance recruitment efficiency.

1.7.3 User Characteristics

The intended users of the system typically possess basic digital literacy and access to internet-enabled devices. Students are expected to have foundational knowledge in their respective fields, while employers require the ability to define task requirements and evaluate outcomes. The platform is designed with a user-friendly interface to accommodate

users with varying levels of technical expertise, ensuring accessibility and ease of use.

1.7.4 User Benefits

The system provides multiple benefits to its users by offering a structured and efficient environment for skill development and recruitment. Students gain verified practical experience, employers benefit from efficient talent sourcing, and institutions improve their ability to track and enhance student performance. This multi-user approach ensures that the platform delivers value across different segments of the ecosystem.

1.8 Software Process Model

The development of the Smart AI Micro-Internship Program follows an Agile-based Incremental Software Process Model. This model is suitable for modern web-based systems that require flexibility, continuous improvement, and integration of multiple components such as front-end, back-end, and AI modules. The model allows the system to be developed in phases, where each phase produces a functional part of the system that can be tested and improved iteratively [15].

1.8.1 Process Model Introduction

The Agile Incremental Process Model is a combination of iterative development and incremental delivery. In this model, the system is divided into smaller modules, and each module is developed, tested, and integrated in successive iterations. Unlike traditional models, Agile allows continuous feedback from users and stakeholders, enabling the system to evolve based on changing requirements.

This approach is particularly effective for projects that involve dynamic requirements and complex integrations, as it ensures that development progresses in manageable stages while maintaining system quality and performance.

1.8.2 Justification of Proposing the Process Model

The Agile Incremental model is chosen over traditional models such as the Waterfall model due to its flexibility and adaptability. In the Waterfall model, requirements must be fully defined at the beginning and each phase must be completed before the next begins, which is not suitable for this project as it involves evolving features such as AI-based recommendations and performance analytics. Because the Waterfall model produces working software only at the very end, any misunderstanding in the early requirements would remain hidden until late in the project, when it is expensive and difficult to correct [16].

Other models were also considered before settling on the Agile Incremental approach. The Spiral model offers strong risk management through repeated risk analysis cycles, but its heavy emphasis on formal risk documentation and prototyping is more appropriate for very large, high-budget systems than for a time-bound final year project [17]. A pure prototyping model, on the other hand, is useful for clarifying user interface requirements but does not provide a disciplined structure for delivering and integrating a complete, multi-module system. The Agile Incremental model was selected because it combines the structured, module-by-module delivery of the incremental approach with the flexibility,

continuous feedback, and short feedback loops of Agile, which together fit the evolving and AI-driven nature of this project far better than any single traditional model.

The proposed system requires continuous refinement of algorithms, user interfaces, and system functionalities. The Agile approach allows:

- Early development of functional modules and prototypes.
- Continuous testing and feedback integration.
- Easy handling of changes in requirements.
- Parallel development of multiple system components.

These characteristics make Agile more suitable for the Smart AI Micro-Internship Program, where iterative improvements and user feedback are essential for achieving optimal system performance.

1.8.3 Steps of Process Model

The development process is carried out in a series of iterative steps, where each step focuses on delivering a specific set of functionalities. The main steps of the process model are described below:

- **Requirement Analysis:** Identification and documentation of system requirements, including user needs and system functionalities.
- **System Design:** Development of system architecture, database design, and user interface prototypes.
- **Implementation:** Coding and development of individual modules such as front-end, back-end, and AI components.
- **Testing:** Verification of system functionality through unit testing, integration testing, and performance testing.
- **Integration:** Combining all developed modules into a unified system and ensuring smooth interaction between components.
- **Deployment:** Launching the system on a hosting platform for user access and operation.
- **Maintenance and Improvement:** Continuous monitoring, bug fixing, and enhancement based on user feedback and system performance.

This iterative cycle continues throughout the project timeline, ensuring gradual improvement and successful delivery of a reliable and scalable system.

1.8.4 Iteration and Sprint Plan

Although the overall process is incremental, the day-to-day work of the Smart AI Micro-Internship Program is organised into fixed-length Agile sprints so that progress remains measurable and predictable. Each sprint lasts approximately two weeks and follows the standard Agile ceremonies, namely a sprint planning meeting at the start, short progress check-ins during the sprint, a sprint review with the supervisor at the end, and a brief retrospective to identify improvements for the next sprint. At the start of every

sprint, a prioritised set of user stories is selected from the product backlog and broken down into concrete development tasks that are assigned to individual team members. At the end of the sprint, the completed work is integrated, tested, and demonstrated as a potentially shippable increment of the system. This sprint-based rhythm keeps the team focused, exposes problems early, and ensures that each increment delivers visible and working functionality rather than partially finished code.

The development work is grouped into a sequence of larger iterations, where each iteration bundles two or three related sprints and produces a coherent, demonstrable part of the system. The first iteration establishes the foundation by delivering authentication and profile management, after which later iterations build the task and application workflow, the AI matching engine, and finally the interview, evaluation, and analytics features. This ordering is deliberate: foundational modules that almost every other feature depends upon are delivered first, while more advanced and AI-intensive modules, which benefit from iterative training and refinement, are scheduled once the supporting data and interfaces are already in place. The mapping of iterations to their focus modules, key deliverables, and planned timeline is summarised in Table 1.3.

Table 1.3: Iteration and Sprint Breakdown of the Smart AI Micro-Internship Program

Iteration	**Focus Modules**	**Key Deliverables**	**Timeline**
Iteration 0	Requirements and project setup	Approved requirements, architecture, repository and environment setup	Feb–Mar 2026
Iteration 1	Authentication, Student and Company profile management	Secure sign-up/sign-in, email/OTP verification, role-based profiles	Apr–May 2026
Iteration 2	Task posting, search, Application submission and tracking	Task lifecycle, application workflow, status history	May–Jun 2026
Iteration 3	AI-based matching and candidate ranking	FastAPI matching service, match scores, recommendations	Jul–Aug 2026
Iteration 4	Interview scheduling, evaluation, feedback, analytics, certificates	Interview module, ratings, dashboards, verified certificates	Aug–Sep 2026
Iteration 5	Integration and testing	Fully integrated system, test reports, bug fixes	Sep–Nov 2026
Iteration 6	Deployment and documentation	Production deployment, user manual, final report	Nov–Dec 2026

A key advantage of this sprint and iteration structure is that the most valuable and high-risk functionality is validated as early as possible. For example, the AI matching engine in Iteration 3 can be tested against real student and task data produced by Iterations 1 and 2, allowing its scoring weights and recommendations to be refined over

several sprints rather than being built blindly at the end. Likewise, integration is treated as a continuous activity that begins within each iteration instead of being deferred to a single risky phase, which reduces the chance of late surprises. By the time the dedicated integration and testing iteration begins, most modules have already been connected and exercised, so this stage concentrates on end-to-end validation and stabilisation. This disciplined yet flexible approach allows the team to respond to changing requirements while still maintaining a clear and trackable path toward the final deliverable.

1.8.5 Advantages of the Model

The Agile Incremental model provides several advantages for this project. It supports flexibility in handling changing requirements, enables faster delivery of functional components, and promotes continuous testing and quality assurance. Additionally, it facilitates better collaboration among team members and stakeholders, ensuring that the system aligns with user expectations and project objectives.

1.8.6 Applicability to the Proposed System

The selected process model is particularly suitable for the Smart AI Micro-Internship Program as it involves multiple interdependent modules such as user management, AI-based recommendation, task management, and analytics. The incremental approach allows each module to be developed and tested independently before integration, ensuring system reliability and scalability. This approach also supports the integration of AI models, which may require iterative training, evaluation, and refinement.

Chapter 2

SOFTWARE REQUIREMENTS SPECIFICATION

Before any module of the Smart AI Micro-Internship Program was designed or built, the team needed to agree in writing on what the system must do and what quality standards it must meet. This chapter records those agreements as a Software Requirements Specification — a structured document that captures every functional capability and every measurable quality constraint the final platform must satisfy.

Getting requirements pinned down early pays dividends throughout development. When preparing this chapter, writing precise requirements exposed several ambiguities that would have caused costly rework if they had only surfaced during coding. For instance, it was not initially obvious whether the AI matching score should be visible to students as well as companies, or whether a company account on the free tier should be permitted to post an unlimited number of tasks. Working through the SRS forced the team to resolve these questions deliberately and record the decisions for reference throughout the project.

The requirements documented here were gathered through interviews with potential users, a review of comparable platforms, and a study of related academic literature. They are organised into two broad categories: functional requirements, which describe the specific features the platform must provide, and non-functional requirements, which set measurable standards for performance, security, usability, and reliability. Both categories are essential — a system that does the right things but does them slowly, insecurely, or inconsistently would ultimately fail its users just as surely as one that simply does the wrong things.

2.1 Introduction

The software requirements phase is an important step in system development where the current system is analyzed from different aspects. In this phase, all existing data and processes are carefully studied to understand how the current system works [18].

The manual data and procedures used in the existing system are examined in detail. This helps in identifying the limitations, issues, and inefficiencies present in the current approach. By analyzing this information, it becomes easier to define the requirements for the new system.

This phase ensures that the proposed system is designed in a way that overcomes the problems of the existing system and provides a more efficient, reliable, and user-friendly solution.

2.1.1 Document Scope

A Software Requirements Specification (SRS) document defines the complete set of requirements for a software system. It describes the system's functionalities, user interactions, constraints, and expected behavior. The purpose of this document is to provide a clear understanding of what the system is supposed to do, ensuring that all stakeholders have a common reference before development begins [19].

In the context of the Smart AI Micro-Internship Program, this SRS document outlines the requirements for developing a web-based platform that connects students with organizations through structured micro-internships. It defines system features such as user management, task assignment, AI-based recommendations, performance evaluation, and certification. This document serves as a guideline for designing, developing, and testing the system in an organized and efficient manner.

2.1.2 Audience

An SRS document is generally intended for all individuals involved in the software development process, including developers, testers, project managers, and stakeholders. It helps them understand the system requirements, expected functionalities, and overall system behavior.

For this project, the intended audience includes developers who will implement the system, testers who will validate the system against the requirements, and project supervisors who will evaluate the design and progress. Additionally, stakeholders such as educational institutions and organizations can use this document to understand how the Smart AI Micro-Internship Program will fulfill their needs.

2.2 Functional Requirements

Functional requirements describe the core features and operations that a system must perform. These requirements define how the system behaves, how users interact with it, and what services the system provides. They ensure that all necessary functionalities are clearly defined before implementation [20].

For the Smart AI Micro-Internship Program, the functional requirements are derived from the key modules of the system such as user management, task handling, AI-based matching, evaluation, and certification. These modules work together to provide a complete platform for students and organizations to interact efficiently.

2.2.0.1 Company Profile Management The system shall allow organizations to create, update, and manage their company profiles within the platform. Each profile shall store essential details such as the company name, industry, description, location, contact information, and the opportunities currently being offered. Organizations shall be able to edit this information at any time so that it always remains accurate and up to date. A clearly presented company profile helps students understand the nature of an employer before applying for its tasks. This module therefore improves transparency and builds trust between organizations and students, while giving employers a single place to manage their presence on the platform.

2.2.0.2 Application Tracking The system shall allow students to apply for available tasks and to track the status of each of their applications in real time. Every application shall progress through clearly defined stages such as submitted, under review, shortlisted, accepted, or rejected, and the student shall be notified whenever the status changes. Employers shall be able to view, filter, and manage all applications submitted for their tasks from a single dashboard. The system shall also maintain a complete history of status changes so that both parties can see how an application has progressed over time. This transparent tracking reduces confusion and keeps students informed throughout the selection process.

2.2.0.3 Interview Scheduling The system shall provide functionality for organizations to schedule interviews with shortlisted students directly within the platform. Employers shall be able to set the date, time, duration, and mode of the interview, such as video, phone, or on-site, along with the relevant meeting details. Both the student and the employer shall receive timely notifications and reminders about each scheduled interview. The system shall also allow interviews to be rescheduled or cancelled within defined limits so that unexpected changes can be handled gracefully. By integrating scheduling into the platform, this module reduces dependency on external communication tools and keeps the recruitment workflow organized.

2.2.0.4 Mentor Assignment The system shall allow experienced professionals or supervisors to be assigned as mentors to students during their micro-internships. Mentors shall be able to guide students, monitor their work, and offer advice while tasks are being completed. The system shall make it easy to assign a mentor to a specific student and task and to keep a record of these assignments. Through this support, students receive professional guidance instead of working in isolation, which improves the overall quality of their learning experience. This module strengthens the educational value of the platform by combining real industry tasks with structured mentorship.

2.2.0.5 Progress Tracking The system shall continuously track the progress of students as they work through their assigned tasks. It shall record key indicators such as milestones reached, submission status, time spent, and overall performance throughout the internship. Both mentors and organizations shall be able to view progress reports to understand how a student is performing. The system shall update this information regularly so that any delays or difficulties can be identified early. By providing a clear view of ongoing work, this module promotes accountability and allows timely support whenever a student needs help.

2.2.0.6 Feedback System The system shall allow employers and mentors to provide structured feedback on a student's performance after tasks or interviews are completed. This feedback may include written comments, ratings, and specific suggestions for improvement. Students shall be able to view the feedback so that they can understand their strengths and the areas that need further development. The system shall store feedback against the relevant task so that it becomes part of the student's overall performance record. By encouraging constructive and continuous feedback, this module supports professional growth and helps students improve with each new opportunity.

2.2.0.7 Payment Integration The system shall support secure payment processing for tasks and internships that offer compensation. It shall integrate with a trusted digital payment gateway to handle transactions between organizations and students. The

system shall record payment details and statuses so that both parties have a clear and transparent record of every transaction. Sensitive financial information shall be handled securely and in line with standard payment-security practices. By enabling fair and reliable compensation, this module encourages greater participation from both students and organizations.

2.2.0.8 Document Verification The system shall allow important documents, such as certificates, credentials, and company verification papers, to be reviewed and verified. Administrators shall be able to check submitted documents and approve or reject them based on their authenticity. The verified status shall be clearly indicated so that other users can trust the information presented on a profile. This process helps prevent fraudulent accounts and ensures that only legitimate organizations and genuine student credentials appear on the platform. As a result, this module strengthens the overall credibility and safety of the system.

2.2.0.9 AI-Based Matching The system shall use an intelligent matching engine to connect students with the tasks that best fit their abilities. The engine shall analyze each student's skills, experience, interests, and profile details and compare them with the requirements of the available tasks. Based on this analysis, the system shall generate a match score and recommend the most relevant tasks to each student and the most suitable candidates to each employer. The system shall also present the main reasons behind each recommendation so that the results are transparent and easy to understand. By automating this process, the module reduces manual searching and increases the likelihood of successful task completion through better compatibility.

2.2.0.10 Structured Micro-Internship Model The system shall provide a clear and structured workflow for every micro-internship offered on the platform. This workflow shall guide each opportunity through well-defined stages such as task posting, application, candidate selection, task assignment, submission, evaluation, and completion. Each stage shall have defined actions and responsibilities for both students and organizations so that expectations remain clear at all times. The system shall ensure that an internship cannot skip the required steps, which keeps the process consistent and fair for everyone involved. This structured model distinguishes the platform from simple job listings by turning each task into a complete and guided learning experience.

2.2.0.11 Automated Evaluation The system shall automatically evaluate student submissions against predefined criteria once a task is completed. It shall generate scores and structured feedback without requiring fully manual assessment, which improves consistency and saves time for employers and mentors. The evaluation results shall be recorded and linked to the student's performance history and certificates. Where appropriate, reviewers shall still be able to adjust or confirm the automated outcome to ensure fairness. By standardizing assessment, this module provides a reliable and objective measure of each student's practical abilities.

2.2.0.12 Performance Analytics Dashboard The system shall provide an analytics dashboard that presents performance information in a clear and visual form. For students, the dashboard shall display their progress, scores, completed tasks, and skill improvements over time. For organizations, it shall summarize candidate performance and help identify high-performing students. The system shall update these analytics as new data is recorded so that the information always reflects the most recent activity. By

turning raw data into meaningful insights, this module supports data-driven decisions for both students and employers.

2.2.0.13 Verified Certificates The system shall generate verifiable digital certificates for students upon the successful completion of tasks and internships. Each certificate shall include details such as the task completed, the issuing organization, and the date of completion. The system shall make these certificates easy to share and verify so that they can be trusted by future employers. Because they are based on real, evaluated work rather than self-reported claims, the certificates serve as credible proof of a student's skills and achievements. This module adds lasting value to the platform by helping students build a recognized record of practical experience.

2.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of a system. These requirements focus on how the system performs rather than what the system does. They include aspects such as performance, security, usability, reliability, and scalability, which are important for ensuring a smooth and efficient user experience [16].

For the Smart AI Micro-Internship Program, non-functional requirements ensure that the system is reliable, secure, and easy to use for all users including students, employers, and administrators. These requirements help in maintaining system quality and ensure that the platform performs efficiently under different conditions.

2.3.0.1 Performance Requirements The system shall respond to user actions quickly and within an acceptable time under normal operating conditions. It shall be able to handle multiple users interacting with the platform at the same time without noticeable delays. Pages, dashboards, and search results shall load efficiently so that users can complete their tasks smoothly. The system shall also process heavier operations, such as AI-based matching and report generation, within a reasonable time. Maintaining good performance is essential for keeping students and organizations engaged and satisfied with the platform.

2.3.0.2 Security Requirements The system shall ensure that all users are securely authenticated before they can access protected features. It shall enforce proper authorization so that each user can only perform the actions and view the data permitted for their role. Sensitive information, including passwords, personal details, and payment data, shall be protected through encryption and secure communication protocols such as HTTPS. The system shall also guard against common security threats and unauthorized access attempts. Strong security is critical for protecting user privacy and maintaining trust in the platform.

2.3.0.3 Usability Requirements The system shall provide a simple, clean, and user-friendly interface that is easy to learn and use. Users with only basic technical knowledge shall be able to navigate the platform and complete common tasks without confusion. The interface shall follow a consistent design, provide clear feedback for user actions, and display helpful messages when errors occur. Important features shall be easy to locate so that users do not feel overwhelmed. Good usability reduces the learning curve and encourages students and organizations to use the platform regularly.

2.3.0.4 Reliability Requirements The system shall operate consistently and perform its functions correctly without frequent failures. It shall remain stable even when many users are active or when large amounts of data are being processed. In the event of an error or crash, the system shall recover quickly and protect any work that was in progress. The system shall also handle invalid input gracefully instead of breaking unexpectedly. High reliability ensures that users can depend on the platform whenever they need it.

2.3.0.5 Scalability Requirements The system shall be able to handle a growing number of users, tasks, and records without a significant drop in performance. Its architecture shall be designed so that additional resources can be added as demand increases. Computationally heavy components, such as the AI matching service, shall be able to scale independently from the rest of the system. The platform shall continue to perform well as more organizations and students join over time. This scalability ensures that the system can support future growth and expansion without requiring a major redesign.

2.3.0.6 Maintainability Requirements The system shall be designed in a modular and well-organized way so that it can be easily understood and updated. Each module shall be separated by responsibility so that changes in one part have minimal impact on the others. Developers shall be able to fix bugs, improve features, and add new functionality without disturbing existing behavior. The code shall follow consistent standards and be supported by clear documentation to make future maintenance easier. Good maintainability reduces long-term effort and helps the platform evolve smoothly over its lifetime.

2.3.0.7 Compatibility Requirements The system shall be compatible with the major modern web browsers, including Google Chrome, Mozilla Firefox, and Microsoft Edge. It shall function correctly across different devices such as desktops, laptops, tablets, and mobile phones. The interface shall be responsive so that it adapts to various screen sizes while keeping its content readable and usable. The system shall provide a consistent experience regardless of the platform or device a user chooses. This broad compatibility ensures that all users can access the platform conveniently from their preferred environment.

2.3.0.8 Availability Requirements The system shall be available to users at all times except during planned maintenance windows. It shall be deployed in a stable hosting environment that minimizes unexpected downtime. When maintenance is required, users shall be informed in advance wherever possible so that disruption is reduced. The system shall be able to restore normal operation quickly after any interruption. High availability ensures continuous access for students and organizations, which is important for a platform that may be used across different time zones.

2.3.0.9 Data Integrity Requirements The system shall ensure that all data stored in the database remains accurate, complete, and consistent at all times. It shall validate inputs before saving them so that incorrect or malformed data is not accepted. Relationships between records, such as those linking students, tasks, and applications, shall be maintained correctly to avoid orphaned or conflicting data. The system shall also protect against accidental data loss and unauthorized modification through proper access controls and regular backups. Maintaining data integrity is essential for producing reliable reports, evaluations, and analytics.

2.3.1 Software Quality Attributes

Software quality attributes define the overall quality and performance standards that a system must meet. These attributes describe how well the system performs its functions and ensure that it is reliable, secure, and efficient. They play an important role in determining the usability and effectiveness of the system [18].

For the Smart AI Micro-Internship Program, these quality attributes ensure that the platform provides a smooth, secure, and reliable experience for all users. Since the system involves multiple users and real-time interactions, maintaining high quality standards is essential for its success.

2.3.1.1 Performance Performance describes how quickly and efficiently the system responds to user actions and processes its workload. For the Smart AI Micro-Internship Program, good performance means that pages, dashboards, and search results load quickly and that operations such as task submission and AI-based recommendations complete without noticeable delay. This is supported by efficient database queries, indexed lookups, and a separate service that handles the heavier matching computations. Performance is measured through response times and through the system's ability to serve many users at the same time without slowing down. Maintaining strong performance keeps the platform responsive and encourages students and organizations to use it regularly.

2.3.1.2 Reliability Reliability refers to the system's ability to operate correctly and consistently over time without unexpected failures. For this platform, reliability means that core actions such as logging in, applying for tasks, and generating certificates work dependably every time they are used. The system is built to handle invalid input gracefully and to recover quickly if an error occurs, so that users do not lose their work. Reliability is reflected in a low rate of failures and in the platform remaining stable even under heavier load. A reliable system builds user confidence, since students and organizations can trust that the platform will behave as expected.

2.3.1.3 Security Security is the quality attribute concerned with protecting the system and its data from unauthorized access and misuse. In the Smart AI Micro-Internship Program, security is achieved through secure authentication, role-based authorization, password hashing, and encrypted communication over HTTPS. Sensitive information such as personal details and payment data is handled carefully so that only authorized users can access it. The system is also designed to resist common threats such as unauthorized requests and data tampering. Strong security protects user privacy and is essential for maintaining trust in a platform that stores personal and professional information.

2.3.1.4 Usability Usability measures how easy and pleasant the system is to learn and use for its intended users. For this platform, usability means that students, organizations, and administrators can complete their tasks through a clear, consistent, and intuitive interface. The design provides helpful feedback, simple navigation, and understandable error messages so that even users with limited technical experience can operate it confidently. Usability can be assessed through user feedback and through the ease with which new users complete common actions such as registering or applying for a task. High usability reduces the learning curve and increases overall satisfaction with the platform.

2.3.1.5 Scalability Scalability is the system's ability to continue performing well as the number of users, tasks, and records grows. The Smart AI Micro-Internship Program is designed with a modular, service-oriented architecture so that additional resources can be added as demand increases. Heavier components, such as the AI matching service, can be scaled independently from the main application to handle larger workloads. This ensures that the platform remains responsive even as more organizations and students join over time. Good scalability allows the system to grow smoothly without requiring a complete redesign of its core.

2.3.1.6 Maintainability Maintainability describes how easily the system can be understood, corrected, and improved after it has been built. In this project, maintainability is supported by organizing the code into clear, modular components, each with a well-defined responsibility. This structure allows developers to fix issues or add new features in one area without unintentionally affecting the others. Consistent coding standards and supporting documentation further reduce the effort required to maintain the system. High maintainability lowers long-term costs and helps the platform evolve as new requirements emerge.

2.3.1.7 Availability Availability refers to the proportion of time that the system is running and accessible to its users. The Smart AI Micro-Internship Program aims to remain available at all times, with downtime limited to short, planned maintenance windows. It is intended to run in a stable hosting environment that minimizes unexpected outages and can restore service quickly after any interruption. Availability is commonly expressed as a percentage of uptime, and the platform targets a high level of continuous operation. Strong availability is important because the system may be accessed by users in different time zones who expect it to be reachable whenever they need it.

2.3.2 Other Non-Functional Requirements

Other non-functional requirements include additional constraints and conditions that a system must satisfy apart from performance and usability. These requirements cover areas such as legal compliance, platform compatibility, and operational constraints that ensure the system functions properly within its environment [19].

For the Smart AI Micro-Internship Program, these requirements ensure that the system operates within legal boundaries, supports multiple platforms, and provides a consistent experience to users across different environments.

2.3.2.1 Legal Requirements The system shall comply with all applicable data protection and privacy laws that govern the collection and processing of personal information. User data shall be handled securely, and clear consent shall be obtained before any personal information is collected or stored. The system shall also give users appropriate control over their own data, in line with recognized privacy principles. In addition, the content and activities hosted on the platform shall follow accepted ethical and legal standards. Meeting these legal requirements protects both the users and the organization that operates the platform from compliance-related risks.

2.3.2.2 Platform Requirements The system shall be developed as a web-based application that runs on modern web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge. It shall be accessible across a range of devices, including desktops, lap-

tops, tablets, and mobile phones, without requiring any special installation. The interface shall be responsive so that it adapts to different screen sizes while remaining easy to use. Because the application is delivered through the browser, users shall always access the latest version without performing manual updates. These platform requirements ensure that the system is convenient and widely accessible to all intended users.

2.3.2.3 Operational Requirements The system shall operate efficiently under normal working conditions without depending on specialized or expensive hardware. It shall function correctly using standard internet connectivity and typical end-user devices, making it practical for a wide audience. The platform shall be straightforward to deploy and operate, with sensible default configurations for hosting and maintenance. Routine operational tasks, such as monitoring the system and applying updates, shall be possible without disrupting active users. These operational requirements ensure that the system can run smoothly in real-world environments throughout its lifetime.

2.3.2.4 Backup and Recovery Requirements The system shall provide reliable mechanisms for backing up its data so that important information is not permanently lost. Backups shall be performed regularly and stored safely so that they can be used to restore the system whenever needed. In the event of a failure, hardware fault, or accidental data loss, the system shall be able to recover the most recent consistent state with minimal disruption. The recovery process shall aim to bring the platform back to normal operation quickly so that users experience little downtime. These backup and recovery measures protect the integrity of the data and ensure the long-term safety of user information.

2.3.2.5 Ethical Requirements The system shall ensure fairness and transparency in its AI-based recommendations and automated evaluations. The matching and scoring logic shall be designed to avoid unfair bias and to give all students an equal opportunity to be matched with suitable tasks. Wherever an automated decision affects a user, the system shall provide clear and understandable reasons so that the outcome does not appear arbitrary. The platform shall also treat all users respectfully and protect them from discrimination or misuse. Upholding these ethical requirements is essential for building trust in a system that influences students' learning and career opportunities.

2.4 Requirement Gathering Techniques Used

Requirement gathering is an important phase in software development where information is collected to understand user needs and system expectations. Different techniques are used to gather accurate and complete requirements, such as interviews, questionnaires, and observation. These techniques help in identifying system functionalities and user expectations clearly [20].

For the Smart AI Micro-Internship Program, multiple requirement gathering techniques were used to understand the needs of students, employers, and other stakeholders. These techniques helped in collecting relevant information and defining system requirements more effectively.

2.4.1 Technique 1: Interviews

An interview is a direct and interactive requirement gathering technique in which a developer or analyst holds a face-to-face or online conversation with a stakeholder to understand their needs, expectations, and concerns regarding the proposed system. It allows the interviewer to ask open and follow-up questions, explore the reasoning behind each answer, and capture information that would otherwise be difficult to obtain through written forms. The main benefit of interviews is that they provide rich, detailed, and context-specific information directly from real users and subject-matter experts. They also help in building trust between the development team and the stakeholders, which improves cooperation during later project phases. However, interviews also have some disadvantages. They are time-consuming to arrange and conduct, the quality of information depends heavily on the interviewer's communication skills, and responses may sometimes be biased by the personal opinions of individual participants.

For this project, I personally arranged and conducted several interviews during the requirement gathering phase. I first prepared a short list of target participants, including university students, recent graduates, a few teaching assistants, and two professionals working in the software industry. I contacted most of them through university groups, LinkedIn, and personal references, and scheduled short 20 to 30 minute sessions using Google Meet and in-person meetings on campus. Before each interview, I briefly explained the idea of the Smart AI Micro-Internship Program so that the participant understood the context. During the interviews, I took written notes and, with permission, recorded a few sessions to review them later. After each interview, I summarized the main points and highlighted the features and pain points that were mentioned most often, which then directly influenced the functional requirements of the system.

A few sample questions that I used during the interviews are listed below.

- What are the main difficulties you face when searching for internships or short-term practical experience opportunities?
- Which existing platforms have you used for internships or freelancing, and what features did you find most helpful or missing?
- How important is it for you that tasks and internships are matched to your skills automatically using AI, instead of manual searching?
- What kind of feedback or evaluation would help you improve your skills during and after a micro-internship?
- Would verified digital certificates issued after task completion encourage you to use such a platform, and why?
- As a potential employer or mentor, what features would make it easier for you to assign, monitor, and evaluate student tasks online?

2.4.2 Technique 2: Questionnaires

A questionnaire is a structured requirement gathering technique in which a set of predefined questions is shared with a large group of users to collect their opinions, preferences, and experiences in a consistent format. Questionnaires usually contain a mixture of closed-ended questions, such as multiple choice and rating scales, and a few open-ended questions that allow respondents to share additional comments. The main benefits

of questionnaires are that they can reach a large audience in a short amount of time, the responses are easy to compare and analyze statistically, and participants can answer at their own convenience without the pressure of a direct conversation. However, questionnaires also have disadvantages, such as a limited ability to explore deeper reasoning behind answers, a risk of unclear or misunderstood questions, and the possibility of low response rates if the survey is too long or poorly distributed.

For this project, I designed an online questionnaire using Google Forms to reach a wider audience than was possible through interviews. I shared the form through student WhatsApp groups, Facebook groups, and LinkedIn to collect responses from students of different universities, fresh graduates, and a few working professionals. Before distributing the final version, I tested the questionnaire with two classmates to make sure the questions were simple, clear, and not too long. I kept the total number of questions below fifteen so that it could be completed in around five minutes, which helped in getting a better response rate. After collecting the responses, I used the built-in charts of Google Forms and simple Excel summaries to identify the most commonly requested features, the most frequent pain points, and the general interest level in an AI-based micro-internship platform. These findings were then converted into concrete functional and non-functional requirements.

A few sample questions that were included in the questionnaire are listed below.

- How often do you look for internships, part-time tasks, or freelance work related to your field of study?
- On a scale of 1 to 5, how satisfied are you with the existing internship and freelancing platforms you have used?
- Which of the following features would be most valuable to you: AI-based task matching, automated evaluation, performance analytics, verified certificates, or mentor support?
- Would you prefer short, task-based micro-internships over traditional long-term internships, and why?
- How important is it for the platform to provide personalized feedback and skill improvement suggestions after each task?
- Would you be willing to use a web-based platform that connects students with real companies for paid and unpaid micro-tasks?

2.4.3 Technique 3: Observation

Observation is a requirement gathering technique in which the analyst studies how real users interact with existing systems or perform their daily tasks in their natural environment. Instead of asking users what they do, the analyst watches their actual behavior, notes the steps they follow, and identifies the difficulties or workarounds they use. The main benefits of observation are that it provides accurate and realistic information about user behavior, highlights usability issues that users may not mention in interviews, and often reveals hidden requirements that users consider "normal" and forget to describe. However, observation also has some disadvantages. It can be time-consuming, users may behave differently when they know they are being watched, and it is sometimes difficult to observe certain tasks due to privacy or access restrictions.

For this project, I used observation as a supporting technique along with interviews and questionnaires. I personally observed several students and fresh graduates while they used existing platforms such as Internshala, LinkedIn Jobs, Upwork, and Handshake to search for internships and short-term projects. During these sessions, I sat next to the user or shared their screen through Google Meet, and I noted how long they spent on each step, where they got confused, which filters they used, and which buttons or sections they ignored. I also observed how they reacted to irrelevant job recommendations and how they tried to update their profiles. These observations gave me a clear picture of the usability gaps in existing systems and helped me design a simpler, AI-driven workflow in the proposed Smart AI Micro-Internship Program.

A few sample observation points that were recorded during these sessions are listed below.

- How many clicks and how much time are required for a new user to create a complete student profile on existing platforms?
- Which filters and search options do users most commonly apply while looking for internships and tasks?
- How often do users receive job or task recommendations that are clearly not related to their skills or interests?
- At which step of the application process do users become confused or abandon the task completely?
- How do users react when they do not receive any feedback or status update after submitting an application?
- Which features of the existing platforms do users use the most, and which features do they completely ignore?

2.4.4 Technique 4: Literature Review

A literature review is a requirement gathering technique in which existing research papers, articles, books, technical reports, and documentation of similar systems are studied to understand the current state of the art in a particular domain. It helps in identifying existing solutions, commonly used technologies, standard features, open problems, and limitations of previous work. The main benefits of a literature review are that it builds a strong theoretical foundation for the project, reduces the chance of repeating mistakes made in earlier systems, and helps in selecting proven methods and frameworks. However, literature reviews also have some disadvantages. They can be time-consuming, access to high-quality papers sometimes requires paid subscriptions, and the information may be outdated if only older sources are used.

For this project, I carried out a focused literature review to better understand the domains of AI-based recruitment, job recommendation systems, micro-internship models, and online skill evaluation platforms. I searched for relevant papers and articles on Google Scholar, IEEE Xplore, and the ACM Digital Library using keywords such as “AI recruitment”, “resume parsing”, “job recommendation system”, “micro-internship”, and “online skill assessment”. I preferred papers published in the last five to seven years so that the information remained recent and technically relevant. Alongside research papers, I also studied the public documentation and feature pages of platforms such as Upwork,

Internshala, LinkedIn Jobs, Fiverr, and Handshake to compare their features with the proposed system. The findings from this review were then used to refine the project scope, choose suitable AI techniques, and design an improved feature set for the Smart AI Micro-Internship Program.

A few sample sources and topics that were studied during the literature review are listed below.

- Research papers on AI-based resume parsing and automated candidate screening in modern recruitment systems.
- Studies on machine learning based job recommendation and candidate ranking approaches.
- Articles and reports on the growing importance of micro-internships and short, project-based learning.
- Documentation and feature descriptions of popular platforms such as Upwork, Internshala, LinkedIn, Fiverr, and Handshake.
- Reports from organizations such as the World Economic Forum and the International Labour Organization on youth employment and future skills.
- Guidelines from standard software engineering references on requirements engineering and quality attributes of web-based systems.

2.5 Time Frame

The time frame of the requirement gathering phase refers to the duration in which system requirements are collected, analyzed, and finalized. This phase is important because it forms the foundation of the entire development process. Proper time allocation ensures that all user needs are clearly understood before moving to system design and implementation [16].

According to the project development plan, the requirement gathering phase for the Smart AI Micro-Internship Program was carried out from February 2026 to March 2026. During this period, various techniques such as interviews, questionnaires, observation, and literature review were used to collect and analyze system requirements. This phase was completed before the design phase to ensure that all functionalities and user needs were clearly defined in alignment with the overall project timeline.

Chapter 3

SOFTWARE PROJECT PLAN

3.1 Deliverables of the Project

Project deliverables are the tangible outputs and results that are produced during and after the completion of a project. These deliverables help in evaluating the progress and success of the project and ensure that all objectives are achieved as planned [21].

For the Smart AI Micro-Internship Program, the deliverables include all major components developed during the project lifecycle, such as system design, implementation, documentation, and final deployment. These deliverables ensure that the system is complete, functional, and ready for use.

3.1.1 Software System

The main deliverable of the project is the fully functional web-based application of the Smart AI Micro-Internship Program. This system will include all major modules such as student and company profile management, task posting, application tracking, interview scheduling, mentor assignment, progress tracking, AI-based matching, automated evaluation, feedback, performance analytics, payment integration, document verification, and verified certificate generation. Each module will be developed according to the approved design and connected through well-defined interfaces to ensure smooth data flow between the front-end, back-end, and AI components. The system will be tested on different browsers and devices to confirm that it works correctly in a typical web environment. This working software product represents the most important outcome of the project, because it directly demonstrates how the proposed ideas have been translated into a real and usable platform. It will be handed over to the supervisor along with deployment instructions so that it can be demonstrated, evaluated, and used for future improvements.

3.1.2 Project Documentation

The project documentation is another key deliverable and includes all written materials produced during the project lifecycle. This set of documents includes the Software Requirements Specification (SRS), the software design document, the project management plan, testing reports, and the final project report submitted at the end of the semester. Together, these documents describe the purpose of the project, the analysis of existing systems, the functional and non-functional requirements, the architectural and database design, the chosen technologies, and the final results. The documentation is prepared in parallel with the development work so that decisions and changes are captured while they are still fresh. Proper documentation is important for several reasons: it helps the supervisor and evaluation committee understand the work without running the code, it allows new team members to quickly get familiar with the project, and it supports future maintenance and extensions of the system. All documents will be delivered in both editable and PDF formats.

3.1.3 Source Code

The complete source code of the Smart AI Micro-Internship Program will be delivered as part of the final project package. The source code will be organized into clearly named folders for the front-end, back-end, AI modules, database scripts, and configuration files, so that each part of the system can be located and understood easily. The code will follow consistent naming conventions, proper indentation, and modular structure in order to improve readability and maintainability. Where necessary, short comments will be added to explain non-obvious logic, complex algorithms, and important configuration values. The source code will be managed using a version control system such as Git and hosted on a private repository, which allows the team to track changes, collaborate effectively, and restore previous versions if needed. A short README file will also be provided to explain how to set up, build, and run the system locally, making it easier for reviewers to verify the implementation.

3.1.4 Testing Reports

Testing reports are an important deliverable that provide evidence that the developed system has been properly validated against the defined requirements. These reports will describe the testing strategy followed during the project, including the types of testing performed such as unit testing, integration testing, system testing, and user acceptance testing. For each type of testing, the reports will list the test cases, the input data used, the expected output, the actual output, and the final status such as pass or fail. In addition to functional testing, the reports will also cover basic performance checks, security checks, and usability checks to ensure that the system is reliable, safe, and easy to use. Any defects found during testing will be documented along with their severity, the fix that was applied, and the result of re-testing. These reports help in building confidence that the Smart AI Micro-Internship Program works as expected under different scenarios and is ready for real use.

3.1.5 User Manual

A user manual will be prepared as a separate deliverable to help end users understand how to use the Smart AI Micro-Internship Program without requiring technical assistance. The manual will be written in simple language and will include step-by-step instructions supported by screenshots of the actual system. It will cover common tasks such as creating an account, logging in, completing a profile, searching and applying for micro-internships, posting tasks as an employer, scheduling interviews, tracking progress, submitting work, providing and viewing feedback, downloading verified certificates, and managing account settings. The manual will also describe how to recover a forgotten password and how to contact support in case of issues. In addition to the main user manual, a short quick-start guide will be provided for new users who want to start using the system immediately. This deliverable ensures that both students and organizations can adopt the platform with minimal training and reduces the support effort required after deployment.

3.1.6 Final Deployment

The final deployment of the system is the last deliverable of the project and marks the point at which the Smart AI Micro-Internship Program becomes available for real users.

The deployment will be carried out on a suitable hosting environment such as a cloud platform or a university server, depending on the resources approved by the supervisor. Before deployment, the system will go through a release checklist that includes final testing, database migration, environment configuration, and security hardening steps such as enabling HTTPS and setting strong credentials. After deployment, a short smoke test will be performed to confirm that all major features work correctly in the production environment. Access links, admin credentials, and deployment documentation will be handed over to the supervisor along with instructions on how to restart, back up, and update the system in the future. This deliverable ensures that the project is not only completed in theory but is also usable by real students, employers, and mentors.

3.2 Software Project Management Plan

Software project management is concerned with planning, organizing, and controlling the activities involved in developing a software system. It ensures that the system is delivered on time, within the defined schedule, and according to the specified requirements [21].

For the Smart AI Micro-Internship Program, project management focuses on properly managing tasks, resources, and timelines throughout the development process. It ensures that each phase of the project, from requirement gathering to final deployment, is completed efficiently and meets the desired objectives.

3.2.1 Project Planning

The project planning phase of the Smart AI Micro-Internship Program started in February 2026 and formed the foundation for all later activities. During this phase, the team worked together to clearly define the purpose of the project, its target users, the main problems it aims to solve, and the expected outcomes. The overall scope was finalized based on discussions with the supervisor and the initial results of the requirement gathering activities. After that, a detailed work plan was prepared in which the entire project was broken down into smaller and manageable tasks such as requirement collection, system design, module development, integration, testing, and deployment. Each task was then assigned to a specific team member based on their skills and interests, for example, front-end work, back-end APIs, AI models, and documentation. Realistic deadlines were set for each task, and key dependencies between tasks were identified to avoid bottlenecks. This structured planning ensured that every team member knew exactly what to do, when to do it, and how their work contributes to the overall system.

3.2.2 Project Schedule

The overall project schedule of the Smart AI Micro-Internship Program follows a ten-month timeline from February 2026 to December 2026. This timeline was chosen based on the academic calendar, the complexity of the proposed features, and the availability of the team members. The project is divided into several major phases that are arranged in a logical order so that the output of one phase becomes the input of the next. Some phases also overlap with each other to allow parallel work on different modules, which helps in saving time without affecting quality. A high-level view of the schedule is presented below and is later shown in detail through the Gantt chart in this chapter. The schedule

is continuously monitored during weekly team meetings, and small adjustments are made whenever a task takes more or less time than expected, so that the overall project deadline is still met. The major phases of the project are:

- Requirement Gathering (Feb – Mar 2026)
- System Design (Mar – Apr 2026)
- Development Phase (Apr – Sep 2026)
- Integration and Testing (Aug – Nov 2026)
- Final Deployment (Nov – Dec 2026)

This schedule ensures that all phases are completed in an organized manner, allows enough time for testing and improvements, and provides clear checkpoints that can be used to measure progress throughout the project lifecycle.

3.2.3 Team Management

Effective team management is essential for the successful completion of the Smart AI Micro-Internship Program, since the project involves multiple modules that must work together smoothly. The project team is divided into clear roles such as front-end developers, back-end developers, database designers, AI and machine learning engineers, and documentation leads. Each role has specific responsibilities, but team members also support each other whenever a task requires collaboration across modules. Regular communication is maintained through weekly online meetings, a shared group chat for quick updates, and a task board that shows the status of all ongoing work. The project leader is responsible for tracking overall progress, resolving conflicts, and reporting to the supervisor. Common development standards, such as coding style, branch naming, and commit message format, are agreed upon in advance so that the final codebase remains consistent. This structured approach to team management helps in avoiding duplication of work, reducing misunderstandings, and ensuring that the project moves forward at a steady and predictable pace.

3.2.3.1 Milestones Plan A milestones plan defines the key stages of a project and the important goals that must be achieved at each stage. These milestones help in tracking project progress and ensure that development is moving according to the planned schedule [22].

For the Smart AI Micro-Internship Program, milestones are defined based on the project timeline from February 2026 to December 2026. Major milestones include completion of requirement gathering, system design, development of core modules, integration and testing, and final deployment. Each milestone represents a significant achievement in the project lifecycle.

3.2.3.2 Documentation Plan A documentation plan outlines how all project-related documents will be prepared, maintained, reviewed, and updated throughout the development process. Proper documentation is important not only for understanding the current state of the system but also for supporting future maintenance, handover to new team members, and evaluation by the supervisor. For this project, documentation is treated as a continuous activity rather than a final step, which means that each phase of development produces its own set of documents as the work progresses. The documentation set

includes the Software Requirements Specification (SRS), the software design document, class and database diagrams, test plans and test reports, the user manual, and the final project report. A shared cloud folder is used so that all team members can access and update documents in real time, and version numbers are added to each document to track major changes. Before any document is considered final, it is reviewed by at least one other team member to check for accuracy, clarity, and consistency with the actual implementation.

3.2.3.3 Resources Plan A resources plan identifies all the resources required for the successful completion of the project, including human resources, software tools, hardware, and other supporting services. Proper allocation and management of these resources ensures that tasks are completed efficiently and that no part of the project is delayed due to missing tools or skills. For the Smart AI Micro-Internship Program, the human resources include the project team members who are responsible for front-end development, back-end development, AI and machine learning modules, database design, testing, and documentation. The software resources include programming environments such as Visual Studio Code, web frameworks for the front-end and back-end, a database management system, AI libraries, version control tools like Git and GitHub, and communication tools such as Google Meet and a group chat. The hardware resources include personal computers with sufficient RAM and storage, stable internet connections, and free or low-cost cloud services for hosting and testing. These resources are reviewed regularly to make sure they are available when needed and are used effectively throughout the project lifecycle.

3.2.3.4 Quality Plan A quality plan defines the standards, processes, and activities that are used to ensure that a software system meets the required level of quality. It focuses on maintaining reliability, performance, and correctness throughout the development process [18].

For the Smart AI Micro-Internship Program, the quality plan involves continuous testing, validation, and review of each module during development. Different types of testing such as unit testing, integration testing, and system testing are performed to ensure that the system functions correctly. Regular checks are carried out to verify that all requirements are properly implemented and that the system provides a smooth and reliable user experience.

3.2.4 Project Scheduling

Project scheduling refers to the process of planning and organizing project activities over a specific period of time. It defines when each task will start and finish, helping ensure that the project is completed within the given deadline. Proper scheduling helps in managing time efficiently and tracking project progress [21].

For the Smart AI Micro-Internship Program, project scheduling is based on the Gantt chart timeline from February 2026 to December 2026. The project is divided into different phases, and each phase is assigned a specific duration to ensure smooth and timely completion.

3.2.4.1 Requirement Gathering Phase The requirement gathering phase was conducted from February 2026 to March 2026 and served as the foundation for all later work

on the Smart AI Micro-Internship Program. During this period, the team focused on clearly understanding the real needs of students, fresh graduates, employers, and mentors who would use the system. Multiple techniques were used in parallel to gather accurate and complete information, including one-to-one interviews with potential users, online questionnaires distributed through university groups, observation of users interacting with existing platforms, and a detailed literature review of related research papers and commercial systems. The collected information was analyzed, grouped, and converted into a draft list of functional and non-functional requirements, which was then reviewed with the supervisor. This phase also helped the team define the project scope, identify out-of-scope features, and set realistic expectations. By the end of this phase, a clear and approved set of requirements was ready to guide the design and development activities.

3.2.4.2 System Design Phase The system design phase took place from March 2026 to April 2026 and focused on translating the approved requirements into a concrete technical blueprint of the Smart AI Micro-Internship Program. During this phase, the team worked on the overall system architecture, deciding how the front-end, back-end, database, and AI components would be organized and how they would communicate with each other. Detailed database design was carried out, including the identification of main entities such as users, companies, tasks, applications, evaluations, certificates, and feedback, along with their attributes and relationships. Wireframes and low-fidelity user interface layouts were prepared for the main pages of the system, such as the student dashboard, company dashboard, task details page, and analytics screens. Use case diagrams, sequence diagrams, and activity diagrams were also created to describe the behavior of the system under different scenarios. All design artifacts were reviewed by the supervisor before moving to the development phase so that any major issues could be fixed early.

3.2.4.3 Development Phase The development phase was carried out from April 2026 to September 2026 and represents the longest and most effort-intensive part of the project. During this phase, the approved design was converted into working software using modern web and AI technologies. The team developed the front-end interface using a component-based framework to provide a responsive and user-friendly experience across different devices. The back-end services were implemented as a set of well-organized APIs that handle authentication, profile management, task posting, application tracking, evaluation, feedback, and certificate generation. The AI modules, such as skill-based task matching and basic performance prediction, were developed using standard machine learning libraries and trained on sample datasets. Development was carried out in an iterative manner, where each module was first implemented, tested locally, reviewed by another team member, and then integrated with the rest of the system. Regular code reviews and weekly progress meetings were used to maintain code quality and ensure steady progress throughout this phase.

3.2.4.4 Integration and Testing Phase The integration and testing phase was conducted from August 2026 to November 2026 and partly overlapped with the development phase to allow early detection of issues. In the integration part of this phase, all the independently developed modules such as the front-end, back-end APIs, database, and AI components were connected together to form a complete system. Special attention was given to data flow between modules, correct handling of authentication tokens, and

proper error reporting. In the testing part, a structured test plan was followed that covered unit testing for individual functions, integration testing between related modules, system testing of complete user flows, and user acceptance testing with a small group of volunteers. In addition to functional testing, basic performance, security, and usability checks were also carried out to confirm that the system works well under realistic conditions. Any bugs, broken flows, or usability issues that were found during this phase were logged, fixed, and re-tested before moving to deployment.

3.2.4.5 Deployment Phase The final deployment phase took place from November 2026 to December 2026 and marked the point at which the Smart AI Micro-Internship Program moved from a development environment to a production-like environment where real users could access it. During this phase, the team prepared the system for final delivery by completing the final round of testing, finalizing the database schema and seed data, and configuring the hosting environment for proper performance and security. The system was deployed on a suitable server or cloud platform, and critical settings such as domain names, HTTPS certificates, environment variables, and backup schedules were configured carefully. After deployment, smoke tests were run to confirm that all major features worked correctly in the production environment. In parallel with deployment, the team completed the final project report, the user manual, and the demonstration materials needed for evaluation. The project was then formally submitted to the supervisor along with the deployment details, source code repository access, and all related documentation.

3.2.5 Gantt Chart

The Gantt chart shown in Fig. 3.1 provides a comprehensive visual representation of the proposed Smart AI Micro-Internship System development schedule from February 2026 through December 2026. It divides the overall project into distinct phases, each associated with well-defined tasks, timelines, and milestones. The schedule begins with project planning and kick-off activities in February 2026, followed immediately by requirements analysis to establish functional and non-functional specifications.

The design phase, encompassing both UI/UX prototyping and system architecture definition, spans March and April 2026. Subsequently, the development period extends through July 2026 for front-end implementation and through August 2026 for back-end API development. In parallel, the AI/ML model development begins in June and continues until September, ensuring that the recommendation engine and performance prediction components are fully matured prior to integration.

The integration of individual modules occurs between August and October, enabling combined functionality and end-to-end task flow validation. Beginning in September and extending through November, automated testing and quality assurance activities are undertaken to identify defects and refine system behavior. Beta release testing and feedback collection follow in the final quarter of the schedule to incorporate user insights and make iterative improvements.

This phased timeline also reflects the incremental development methodology adopted for the Smart AI Micro-Internship System. By organizing tasks into overlapping but logically dependent stages, the project team can ensure continuous progress while allowing flexibility for refinement and improvement. Early phases focus on establishing

system requirements and architectural foundations, while later stages emphasize module integration, system validation, and user feedback incorporation. Such a structured schedule enables the team to monitor progress effectively, manage potential risks, and maintain alignment with project objectives, ultimately supporting the successful delivery of a reliable and scalable micro-internship platform within the defined development period.

Finally, the project concludes with the deployment of the fully integrated system and completion of technical documentation during November and December 2026. This structured timeline ensures an organized progression of development activities with adequate overlap and dependencies to support iterative refinement and quality delivery.

Fig. 3.1 gives the visual representation of the overall development process on the basis of the tasks, timeline, deadlines, milestones, and dependencies.

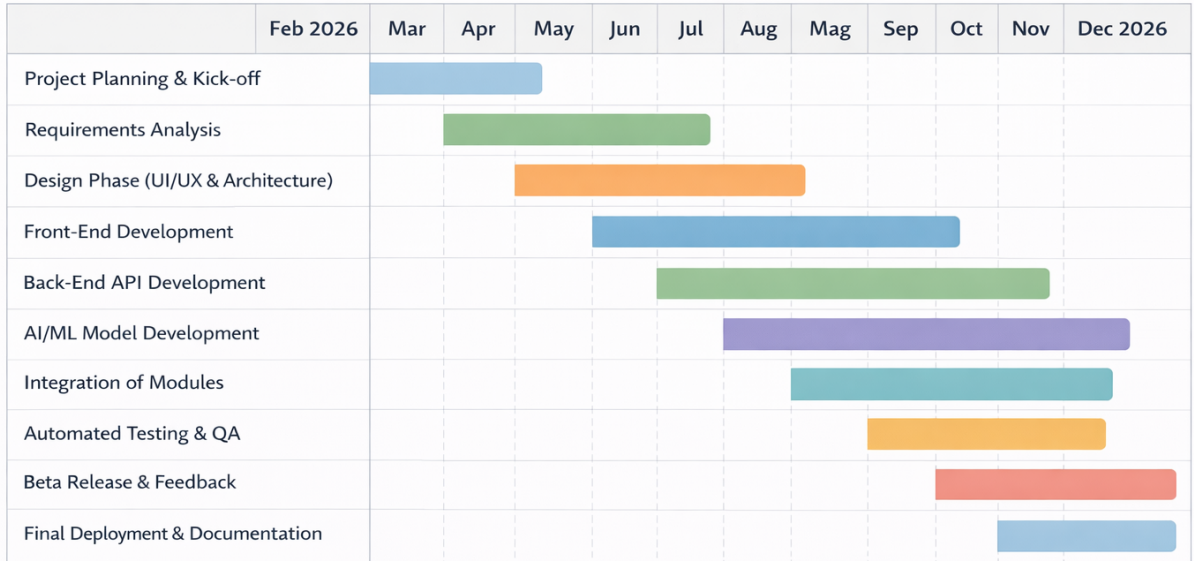

Figure 3.1: Gantt Chart

3.2.5.1 Work Breakdown Structure A Work Breakdown Structure (WBS) is a method used to divide a project into smaller and manageable tasks. It helps in organizing the project work into different levels, making it easier to plan, execute, and monitor each part of the project [22].

For the Smart AI Micro-Internship Program, the project is divided into major phases and each phase is further broken down into smaller tasks. This structure helps the team to clearly understand their responsibilities and ensures that all parts of the system are developed in an organized manner.

3.2.5.2 Requirement Gathering In the Work Breakdown Structure, the requirement gathering branch covers all the smaller activities involved in understanding and documenting user needs for the Smart AI Micro-Internship Program. This includes identifying the main stakeholders such as students, fresh graduates, employers, and mentors, and then selecting suitable techniques to gather information from each group. Sub-tasks under this branch include preparing interview guides, conducting interviews, designing and distributing online questionnaires, observing users interacting with existing platforms,

and performing a literature review of related systems and research papers. After data collection, the information is analyzed and grouped into functional and non-functional requirements. A draft Software Requirements Specification (SRS) document is then prepared and reviewed with the supervisor. Any changes suggested during the review are incorporated, and the final approved requirements become the input for the system design phase.

3.2.5.3 System Design The system design branch of the Work Breakdown Structure includes all the design-related sub-tasks that must be completed before development can begin. It starts with the definition of the overall system architecture, where the team decides how the front-end, back-end, database, and AI modules will be organized and connected. It then moves on to detailed database design, which includes identifying the main entities, attributes, keys, and relationships, and producing entity-relationship diagrams. The user interface design sub-tasks include preparing wireframes and mock-ups for the main screens, defining navigation flows, and finalizing the look and feel of the system. Additional design tasks include preparing use case diagrams, class diagrams, sequence diagrams, and activity diagrams to describe the behavior of the system in detail. Each design artifact is reviewed internally and with the supervisor so that any major issues are identified and corrected early, which helps in reducing rework during development.

3.2.5.4 Development The development branch of the Work Breakdown Structure contains all the coding and implementation sub-tasks that are required to turn the design into a working system. At a high level, it is divided into front-end development, back-end development, database implementation, and AI module development. The front-end sub-tasks include setting up the project with a chosen framework, implementing the main pages such as login, dashboards, task details, and analytics, and connecting these pages to the back-end APIs. The back-end sub-tasks include creating REST APIs for authentication, profile management, task posting, application tracking, evaluation, feedback, and certificate generation. The database sub-tasks include creating tables, writing migrations, and seeding basic data needed for testing. The AI module sub-tasks include preparing the dataset, training a basic model for skill-based task matching, and integrating the model with the back-end. Each module is developed, tested locally, and reviewed before being merged into the main codebase.

3.2.5.5 Integration and Testing The integration and testing branch of the Work Breakdown Structure covers all the sub-tasks needed to combine the individual modules into a complete working system and verify that it behaves correctly. The first set of sub-tasks focuses on integration, where the front-end is connected to the back-end APIs, the back-end is connected to the database, and the AI modules are plugged into the relevant back-end endpoints. Special attention is given to authentication flows, error handling, and data validation across module boundaries. The second set of sub-tasks focuses on structured testing activities, including unit testing of individual functions, integration testing between related modules, and system testing of complete user flows such as registration, task application, and certificate generation. Basic performance testing, security testing, and usability testing are also performed at this stage. All identified defects are logged, fixed, and re-tested so that the system becomes stable and ready for deployment.

3.2.5.6 Deployment and Documentation The deployment and documentation branch represents the final set of sub-tasks in the Work Breakdown Structure and ensures that

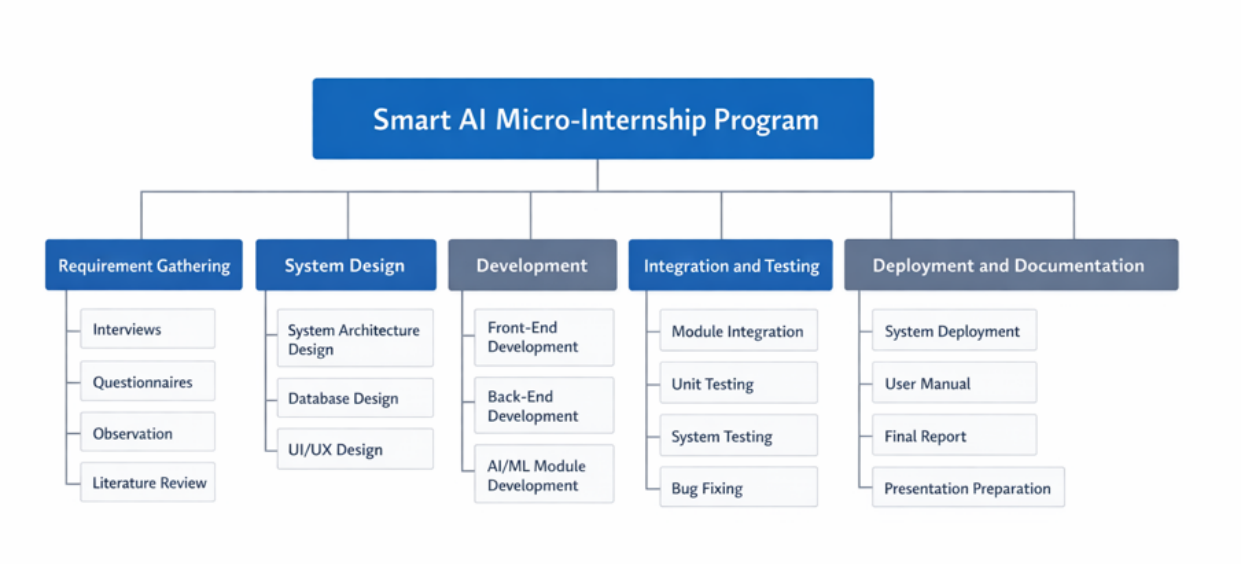

Figure 3.2: Work Breakdown Structure Diagram

the Smart AI Micro-Internship Program is delivered as a complete and evaluable project. The deployment sub-tasks include selecting a suitable hosting environment, configuring the production server or cloud service, setting up the database in the production environment, configuring domain names and HTTPS, and deploying the final version of the application. After deployment, smoke tests are run to confirm that all major features work correctly in the live environment. The documentation sub-tasks include finalizing the Software Requirements Specification, the design document, the test reports, the user manual, and the final project report. In addition, demonstration materials such as slides and screen recordings are prepared for the final presentation. Once all these sub-tasks are completed, the project is formally submitted to the supervisor for evaluation along with the source code repository and deployment details.

3.2.5.7 Critical Path Method The Critical Path Method (CPM) is a project management technique used to identify the sequence of tasks that directly affect the total project duration. It helps in determining the longest path of dependent activities and ensures that any delay in these tasks will delay the entire project [21].

For the Smart AI Micro-Internship Program, CPM is used to identify the most important phases that must be completed on time according to the project schedule. By analyzing task dependencies, the critical path ensures that the project progresses smoothly from requirement gathering to final deployment without unnecessary delays.

3.2.5.8 Critical Activities The critical activities in the Smart AI Micro-Internship Program are the tasks that directly affect the total duration of the project and therefore deserve special attention during planning and execution. These activities include requirement gathering, system design, core module development, integration, system testing, and final deployment. Requirement gathering is critical because it provides the foundation for every other phase; any delay or mistake in this phase affects all later work. System design is critical because it shapes how the front-end, back-end, database, and AI modules are organized and connected. Development of core modules such as user management, task management, AI-based matching, and evaluation is critical because these modules are

needed by almost every other feature. Integration and system testing are critical because they confirm that the complete system works as expected, and deployment is critical because it is the final step that makes the project usable and evaluable. These activities are directly dependent on each other and must be completed in the correct sequence.

3.2.5.9 Importance of Critical Path Identifying the critical path is very important for this project because it helps the team focus their attention and resources on the tasks that have the biggest impact on the overall timeline. Since the Smart AI Micro-Internship Program must be completed within a fixed ten-month window, any delay on a critical task directly pushes the final deadline forward, while delays on non-critical tasks can often be absorbed without major problems. By clearly knowing which tasks lie on the critical path, the project leader can prioritize them during weekly meetings, assign the most experienced team members to them, and track their status more closely. The critical path also helps in making better decisions when unexpected issues appear, for example, whether to move resources from a non-critical task to a critical one. In addition, it supports better time management, smoother coordination between team members, and a more realistic estimation of the final delivery date.

3.2.5.10 Application in the Project In this project, the Critical Path Method is applied in a practical way to monitor the progress of each phase and to ensure that all important tasks are completed within the planned schedule from February 2026 to December 2026. At the beginning of the project, the team identified the sequence of dependent activities that form the critical path, starting from requirement gathering and ending with final deployment. Each critical activity was given a clear start date, end date, and responsible member, and its status is reviewed regularly during team meetings. Whenever a critical task is at risk of being delayed, the team discusses the cause and takes corrective actions, such as splitting the task into smaller parts, adding extra help from other members, or adjusting the scope of less important features. Non-critical tasks, such as secondary documentation or cosmetic UI improvements, are scheduled in parallel so that they do not block the main path. This active use of CPM helps the team to detect problems early, stay on schedule, and deliver a complete and working system on time.

3.3 Managerial Process

3.3.1 Management Objectives and Priorities

Management objectives and priorities define the goals that guide the development process and ensure that the project is completed successfully. These objectives focus on maintaining a productive working environment, meeting deadlines, and delivering software that fulfills user requirements and quality standards [22].

For the Smart AI Micro-Internship Program, the main objective is to develop a reliable and efficient system that meets the needs of students and organizations. The project management focuses on maintaining coordination among team members, completing tasks on time, and ensuring that the system meets all defined requirements.

3.3.1.1 Project Objectives The primary objective of the project is to design and develop a complete web-based platform called the Smart AI Micro-Internship Program

that provides structured micro-internships and AI-based recommendations to students and organizations. The system aims to bridge the gap between academic learning and real industry experience by allowing students to work on short, well-defined tasks posted by real companies. Another important objective is to use artificial intelligence to match each student with the most suitable tasks based on their skills, interests, and past performance, so that the platform becomes more personalized than traditional job boards. The project also aims to support automated evaluation, performance analytics, feedback, and verified certificates, so that student progress can be measured in a fair and trustworthy way. From a technical point of view, the system must be functional, user-friendly, responsive on different devices, and capable of handling multiple users efficiently without major performance issues. These objectives are kept in mind during every phase of development so that the final product aligns with the original vision of the project.

3.3.1.2 Quality Priorities The project places a strong priority on delivering high-quality software that meets both user expectations and standard software engineering practices. Quality is not treated as a single activity at the end of the project, but as a continuous concern that runs through requirement gathering, design, development, testing, and deployment. The main quality priorities include system reliability, usability, performance, maintainability, and security, because these attributes directly affect how users experience the Smart AI Micro-Internship Program in real use. To support these priorities, the team follows consistent coding standards, uses version control for every change, performs code reviews, and writes automated and manual tests for the main features. Bugs found during testing are fixed quickly and re-tested, and any changes in requirements are documented properly before being implemented. The supervisor is also involved at important checkpoints to review both the technical quality and the alignment with project goals. This disciplined approach helps in avoiding rework, reducing technical debt, and producing a final system that is stable, trustworthy, and easy to improve in the future.

3.3.1.3 Time Management Priorities Time management is a key priority in this project because the entire work must be completed within a fixed academic window from February 2026 to December 2026. To handle this constraint, the team has divided the project into clear phases and smaller tasks, each with its own start date, end date, and responsible member. A project calendar and a Gantt chart are used to visualize the schedule and to show how tasks depend on each other, while a simple task board is used to track daily progress. Weekly team meetings are held to review completed work, discuss any delays, and adjust the plan for the following week if needed. Whenever a task takes longer than expected, the team decides either to reduce the scope of less important features, to move extra help to the critical task, or to reschedule non-critical activities. Important milestones such as the requirements review, design review, mid-project demo, and final submission are treated as hard deadlines that must not be missed. This continuous focus on time management helps the team to maintain steady progress and to deliver the final system on schedule.

3.3.1.4 User Satisfaction User satisfaction is considered one of the most important priorities of the Smart AI Micro-Internship Program, because a useful system is only valuable if its intended users actually enjoy using it. To achieve this goal, the team has followed a user-centered approach from the very beginning of the project. During requirement gathering, real students, fresh graduates, and a few professionals were in-

terviewed and surveyed so that their actual needs, pain points, and expectations could directly influence the feature set of the system. During design, simple and clean user interface layouts were created with a focus on easy navigation, minimal clicks, and clear labels. During development, the team regularly uses the system as if they were real users to detect confusing flows and fix them before release. During testing, a small group of volunteer users is invited to try the main features and to share honest feedback about usability, speed, and clarity. Their comments are then used to improve the interface, wording, and workflows. This ongoing focus on user satisfaction helps in making sure that both students and employers have a smooth, reliable, and pleasant experience on the platform.

3.3.2 Assumptions and Constraints

Assumptions and constraints define the conditions under which a software system is developed and operated. Assumptions are the factors that are considered to be true during development, while constraints are the limitations or restrictions that may affect the system design and implementation [16].

For the Smart AI Micro-Internship Program, certain assumptions and constraints are considered to ensure smooth development and implementation of the system. These factors help in defining the boundaries of the project and guide the development process.

3.3.2.1 Assumptions Several assumptions are made during the development of the Smart AI Micro-Internship Program to keep the project scope realistic and achievable within the available time. It is assumed that all target users, including students, fresh graduates, employers, and mentors, have access to internet-enabled devices such as laptops, desktops, or smartphones, and that they also have basic knowledge of using web applications and online forms. It is further assumed that users will provide accurate and valid information while registering, creating profiles, and applying for tasks, so that features such as AI-based matching and verified certification produce meaningful results. The project also assumes that the selected technologies, libraries, and cloud services will remain available and stable during the development period, and that their free or student-tier plans will be sufficient for the expected workload. In addition, it is assumed that participating companies and mentors will respond within a reasonable time and will cooperate with the evaluation and feedback process. These assumptions are documented here so that they can be reviewed and revised if the real situation turns out to be different from what was expected.

3.3.2.2 Technical Constraints The Smart AI Micro-Internship Program is developed under a set of technical constraints that shape the design and implementation decisions throughout the project. The system is planned as a web-based application, which means that it depends on stable internet connectivity and on modern web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge. Users with very old browsers or unreliable internet connections may not be able to enjoy the full experience. The implementation is limited to the selected technologies and tools, such as a specific front-end framework, back-end framework, database, and set of AI libraries. Switching to very different technologies during development is avoided because it would create unnecessary rework and delay. The AI modules are also constrained by the size and quality of the available training data, which means that more advanced deep learning techniques may

not be used in this version of the system. Finally, the use of third-party services such as payment gateways, email providers, and cloud hosting is limited to options that offer free or low-cost plans suitable for a student project.

3.3.2.3 Resource Constraints The project is carried out under clear resource constraints that affect how work is planned and distributed among team members. The most important constraint is time, because the entire system must be designed, developed, tested, and deployed within a fixed ten-month academic schedule from February 2026 to December 2026. Another important constraint is the human resources available for the project, which is limited to the members of the student team, each of whom has a certain amount of experience and weekly availability. Hardware resources are also limited, as the team mostly uses personal laptops with average specifications instead of high-end workstations or dedicated servers. Software resources are mainly restricted to free, open-source, or educational-license tools such as Visual Studio Code, open-source frameworks, and free tiers of cloud services. Because of these constraints, the team focuses on the most important features first, avoids over-engineering, and uses existing libraries and frameworks whenever possible instead of building everything from scratch. This careful use of limited resources helps in keeping the project realistic and deliverable.

3.3.2.4 Operational Constraints The system is designed to operate under a set of operational constraints that reflect a typical web application environment rather than a specialized setup. It must run correctly on standard computing environments and should not require any unusual hardware, expensive servers, or custom network configurations. Users should be able to access the system using normal internet connections and common web browsers on desktops, laptops, tablets, and smartphones. The system is expected to remain available during typical working hours with only short and planned maintenance windows, and it should recover quickly in case of minor failures such as temporary connectivity issues. Operational procedures such as regular database backups, basic monitoring of errors and uptime, and routine updates of dependencies are also considered, so that the platform continues to work reliably over time. In addition, the system must respect the privacy of user data, handle personal information carefully, and follow general security practices such as using HTTPS, strong passwords, and proper access control for different user roles.

3.3.2.5 Dependencies The Smart AI Micro-Internship Program depends on several external technologies and services to work correctly, and these dependencies are clearly identified during planning so that they can be managed throughout the project. The main dependencies include the chosen web frameworks for the front-end and back-end, the database management system used to store user, task, and evaluation data, and the AI and machine learning libraries used for skill matching and basic performance prediction. The system also depends on third-party services such as email providers for notifications, cloud hosting for deployment, and possibly payment gateways for paid tasks. Version control and collaboration depend on tools such as Git, GitHub, and Google Drive, which are used daily by the team. Any major change, outage, or policy update in these external components can directly affect the project, so the team monitors their status and keeps backup options in mind whenever possible. Proper management of these dependencies is important to make sure that the system remains stable, maintainable, and easy to deploy throughout its lifecycle.

3.4 Project Risk Management

No software project of meaningful complexity runs perfectly from start to finish. Team members fall ill, third-party libraries release breaking changes, and features turn out to be harder to implement than initially estimated. For the Smart AI Micro-Internship Program — which combines a web front-end, a Node.js/Express back-end, a Python AI matching service, and a fixed academic deadline — the question was never whether problems would arise, but how quickly the team could detect them and respond.

Our approach was to treat risk management as an ongoing team habit rather than a one-time exercise at project kickoff. Throughout development, the team maintained a shared risk register, held brief weekly reviews to update it, and ensured that each identified risk had a named owner responsible for monitoring it and executing the agreed response if it materialised [17].

3.4.1 Risk Management Plan

The risk management plan sets out the ground rules the team follows when handling uncertainty: who identifies risks, how they are recorded, how likelihood and impact are rated, and what categories of response are considered appropriate [21]. For this project the plan was kept deliberately lightweight so that maintaining it would not compete with actual development time, while still being rigorous enough to guard against the most common project failures — missed milestones, integration breakdowns, and data loss.

In practice, the plan revolves around a simple shared risk register that the team reviews each week. Any team member can add an entry; the project lead is responsible for ensuring every entry has a priority rating and an assigned owner before the next sprint begins.

3.4.1.1 Purpose The main purpose of the risk management plan for the Smart AI Micro-Internship Program is to provide a clear and structured approach for handling uncertainty throughout the project lifecycle. Since the project involves several interconnected modules, new technologies, and a fixed academic deadline, there is always a chance that unexpected events may affect the schedule, the quality of the system, or the availability of resources. The risk management plan ensures that such possibilities are not ignored until they cause real problems, but are instead identified early, properly analyzed, and actively managed. It also helps in creating a shared understanding within the team about what can go wrong, how serious each issue would be, and what actions should be taken if the issue actually occurs. Another important purpose of the plan is to give the supervisor and stakeholders confidence that the team is prepared for common challenges in software projects. Overall, the plan aims to protect the project timeline, quality, and user experience from avoidable disruptions.

3.4.1.2 Roles and Responsibilities Clear roles and responsibilities are essential for effective risk management in the Smart AI Micro-Internship Program, because risks cannot be handled properly if no one is accountable for them. The project leader is responsible for maintaining the overall risk register, scheduling regular risk review meetings, and reporting important risks to the supervisor. Each team member is responsible for identifying risks in their own area of work, such as front-end issues, back-end bugs, AI model limitations, or integration problems, and for reporting them as soon as they are

detected. Whenever a risk is identified, it is assigned to a specific owner who is in charge of tracking it and executing the agreed response plan. The team as a whole participates in weekly meetings to discuss new risks, review the status of existing ones, and update priorities if needed. This distribution of responsibilities ensures that risk management is treated as a shared duty rather than a single person's job, and that nothing is missed due to unclear ownership or poor communication.

3.4.2 Risk Management Activities

Risk management activities are the concrete steps followed by the team to handle risks throughout the Smart AI Micro-Internship Program in a structured and repeatable way. These activities are not treated as a one-time exercise at the beginning of the project, but as an ongoing process that runs in parallel with normal development work. The main activities include risk identification, risk analysis, risk response planning, rating of likelihood and impact, risk monitoring and control, and overall risk assessment. Each of these activities has its own purpose, inputs, and outputs, and together they form a continuous cycle in which new risks are discovered, existing risks are updated, and closed risks are recorded for future reference. The team uses a simple risk register stored in a shared document to keep track of all relevant information, such as the description of the risk, its category, owner, likelihood, impact, and current status. This structured approach helps the team stay prepared, react quickly to issues, and make informed decisions during the project.

3.4.2.1 Risk Identification Risk identification is the first and one of the most important activities in risk management, because a risk that is not identified cannot be managed. In this project, the team actively looks for possible risks from the early planning stage and continues to update the list throughout development. Risks can come from many different sources, such as technical issues with the chosen frameworks or libraries, integration problems between the front-end, back-end, and AI modules, unexpected changes in requirements, delays caused by exams or personal issues of team members, and limitations in resources such as hardware, internet, or free-tier cloud services. Risks related to data quality, model accuracy, and security are also considered, since the system relies on AI-based matching and handles user information. To identify these risks, the team uses a combination of brainstorming sessions, checklists from standard software engineering references, lessons learned from similar projects, and regular discussions during weekly meetings. All identified risks are recorded in the risk register for further analysis.

3.4.2.2 Risk Analysis Risk analysis is the activity in which each identified risk is carefully examined to understand how serious it is and how urgently it must be handled. For every risk in the register, the team estimates two main factors: the likelihood that the risk will actually occur, and the impact it would have on the project if it does occur. Likelihood and impact are usually rated on a simple scale such as low, medium, and high, which makes it easy to compare different risks and focus on the most important ones. During analysis, the team also thinks about the relationships between risks, because some risks can trigger or amplify others, for example, a delay in one module can lead to a delay in integration and testing. The results of the analysis are added to the risk register along with short explanations so that anyone reading the document later can understand the reasoning. This structured analysis helps the team avoid both overreacting to minor

issues and ignoring serious problems until it is too late.

3.4.2.3 Risk Response Planning Risk response planning is the activity in which the team decides how to handle each significant risk that has been identified and analyzed. For every important risk, a response strategy is chosen based on its likelihood, impact, and the resources available for mitigation. Common strategies include avoiding the risk by changing the plan or design, reducing the risk by taking preventive actions, transferring the risk to a third-party tool or service, or accepting the risk when its impact is low and the cost of mitigation is too high. For example, the risk of losing code due to a laptop failure is reduced by using version control and regular pushes to a remote repository, while the risk of low AI model accuracy is reduced by using well-tested libraries and by keeping realistic expectations about what the AI modules can achieve. Each response plan is assigned to a specific team member, and the actions are tracked until the risk is either resolved or reduced to an acceptable level.

3.4.2.4 Rating Risk Likelihood and Impact Rating risk likelihood and impact is a supporting activity that helps the team to prioritize risks in a consistent and objective way. Each risk in the register is rated on two axes: the first is the probability of the risk occurring during the project, and the second is the effect it would have on the schedule, quality, budget, or user satisfaction if it does occur. For this project, a simple three-level scale of low, medium, and high is used on both axes, which keeps the process practical and easy to update during weekly meetings. The combination of likelihood and impact produces an overall risk level, so that the team can clearly distinguish between critical risks that need immediate attention, moderate risks that need regular monitoring, and minor risks that can be accepted as they are. This rating is reviewed and adjusted whenever the situation changes, for example, when a planned mitigation action is completed or when new information becomes available. The resulting priorities guide how the team spends its limited time and energy on risk-related activities.

3.4.2.5 Risk Monitoring and Control Risk monitoring and control is the activity that ensures the risk management plan does not become outdated or forgotten during the busy development phase. Throughout the project, the team continuously watches for signs that existing risks are becoming more likely or more serious, and also looks for new risks that may appear as the system evolves. This is done through regular weekly meetings, quick daily check-ins, and informal conversations whenever someone notices a warning sign, such as a task taking unusually long or a third-party service becoming unstable. When a risk is triggered, the team executes the planned response actions and records what happened in the risk register so that similar situations can be handled faster in the future. If a response plan does not work as expected, the team revises it and tries a different approach. Risks that have been successfully resolved are marked as closed, while new ones are added to the register, keeping the list accurate and relevant at all times.

3.4.2.6 Risk Assessment Risk assessment is the overall evaluation of how effectively risks are being managed in the Smart AI Micro-Internship Program and whether additional actions are required. While risk identification and analysis focus on individual risks, risk assessment takes a step back and looks at the complete picture of the project health from a risk perspective. During assessment, the team reviews the risk register as a whole, checks how many risks are currently active, how many have been resolved, how

many new ones have appeared, and whether there are any patterns that suggest deeper problems. For example, if several risks are related to the AI modules or to integration, it may indicate the need for extra testing or a small redesign of those areas. The assessment is usually carried out at key checkpoints, such as after the design phase, at the mid-project demo, and before final deployment. Its results are shared with the supervisor and used to update the risk management plan if necessary, so that the final system is delivered in a stable and reliable state.

Chapter 4

FUNCTIONAL ANALYSIS AND MODELING

Having agreed on what the Smart AI Micro-Internship Program must do in the previous chapter, the next step is to model how that behaviour looks from the outside and how information flows through the system — without yet committing to any specific technology or implementation detail. This chapter uses three complementary modelling techniques to achieve that. Use case models describe who interacts with the platform and what each actor expects to accomplish. An entity-relationship diagram captures the data entities the system must store and the relationships that bind them together. Data-flow diagrams trace how data moves from one process to another, making the system's information architecture explicit. Taken together, these models form the analytical foundation on which every design decision in Chapter 5 is grounded [23].

4.1 Use Case Modeling

The use case model captures every meaningful interaction between an external actor and the platform, expressing each interaction in plain language that both the development team and non-technical stakeholders can follow [24]. Three human actors drive the main functionality of the Smart AI Micro-Internship Program. The **Student** searches for tasks, applies to them, and tracks their progress. The **Company** (employer) posts tasks, reviews applicants, and manages the hiring pipeline. The **Administrator** oversees the platform, verifies company accounts, and maintains content integrity. A fourth, system-level actor — the **AI Matching Service** — operates automatically in the background, computing a compatibility score each time a student applies for a task and producing ranked candidate lists for company review. The subsections below first summarise each actor's goals as user stories, then expand the most critical interactions into complete use case descriptions with preconditions, step-by-step flows, and alternative paths for error handling.

4.1.1 User Stories

A user story is a short, simple description of a feature told from the point of view of the person who desires that feature, typically following the format "As a [role], I want [goal] so that [benefit]". User stories keep the focus on the value delivered to the user rather than on technical detail, and they are well suited to the incremental and agile development approach adopted for this project [25]. Each story is small enough to be planned, implemented, and tested within a single iteration, which makes progress easy to track across sprints. The user stories below were derived directly from the functional requirements and were used to plan the development of the core modules of the Smart AI Micro-Internship Program. They are grouped by actor so that the responsibilities of each role remain clear and traceable to the corresponding use cases.

Table 4.1: Student User Stories

ID	User Story
US-01	As a student, I want to register and verify my account so that I can securely access the platform.
US-02	As a student, I want to build a detailed profile with my skills, education, experience, and projects so that companies can evaluate me fairly.
US-03	As a student, I want to receive AI-recommended tasks that match my skills so that I do not have to search manually.
US-04	As a student, I want to search and filter available tasks so that I can find opportunities relevant to my interests.
US-05	As a student, I want to apply for a task with a cover letter and resume so that I can be considered by the company.
US-06	As a student, I want to track the status of my applications so that I always know whether I am shortlisted, accepted, or rejected.
US-07	As a student, I want to be notified about scheduled interviews so that I can attend them on time.
US-08	As a student, I want to earn verified certificates after completing tasks so that I can prove my skills to future employers.

4.1.2 Individual Actor Use Cases

Individual actor use cases describe, in a structured form, each significant interaction between a single actor and the system, including the conditions that must hold before the interaction and the result produced afterwards. Documenting use cases at this level of detail clarifies the system boundary, exposes alternative and exceptional flows, and provides a direct basis for the design of screens and the writing of test cases [24]. For the Smart AI Micro-Internship Program, the complete set of use cases is grouped by the three primary actors, and the most critical use cases are then expanded into full use case descriptions. The summary in Table 4.4 lists the principal use cases of each actor, after which selected high-priority use cases are described in detail. This organisation ensures that every functional requirement identified earlier is represented by at least one traceable use case.

The use case in Table 4.5 describes how a student applies for a task. This is one of the central interactions of the platform because it connects student profiles with company tasks and triggers the AI matching process. The description records the actors involved, the conditions required before the action can take place, the normal sequence of steps, and the alternative flows that handle error situations. By specifying these flows explicitly, the design and testing teams can ensure that every input is validated and that invalid states are handled gracefully, which directly addresses the need for robust input handling raised during evaluation.

The use case in Table 4.6 describes how a company schedules an interview with a short-listed student. This interaction demonstrates how the system coordinates two parties and enforces business rules such as limits on rescheduling and the prevention of cancellations

Table 4.2: Company (Employer) User Stories

ID	User Story
US-09	As a company, I want to register and complete a verified company profile so that students trust the opportunities I post.
US-10	As a company, I want to post tasks with required skills, budget, and deadlines so that suitable students can apply.
US-11	As a company, I want to view a ranked list of candidates for each task so that I can shortlist the best applicants quickly.
US-12	As a company, I want to review applications and update their status so that I can manage my hiring pipeline.
US-13	As a company, I want to schedule and manage interviews with shortlisted students so that I can evaluate them directly.
US-14	As a company, I want to provide feedback and ratings after interviews so that students can improve and others can trust the results.

Table 4.3: Administrator User Stories

ID	User Story
US-15	As an administrator, I want to manage user accounts so that I can deactivate or remove fraudulent users.
US-16	As an administrator, I want to verify company documents so that only legitimate organisations can post tasks.
US-17	As an administrator, I want to moderate tasks and content so that the platform remains safe and relevant.
US-18	As an administrator, I want to view platform analytics so that I can monitor growth and overall system health.

close to the interview time. Documenting the alternative flows is important here because interview scheduling involves time-sensitive constraints that must be validated to avoid conflicting or invalid bookings. The use case also shows how the application status transitions to `interview_scheduled`, keeping the application lifecycle consistent across the platform.

The use cases above, together with the summary table, fully cover the goals expressed in the user stories. A consolidated use case diagram can be produced from this specification to give a single visual overview of all actors and their interactions; the textual descriptions provided here define exactly which actors, use cases, and relationships that diagram must contain. This traceable mapping from user stories to detailed use cases ensures that no required functionality is missed before the system proceeds to functional and structural modeling.

Table 4.4: Summary of Use Cases by Actor

Actor	Use Cases
Student	Register/Login, Manage Profile, View Recommended Tasks, Search Tasks, Apply for Task, Track Application, Attend Interview, View Certificate
Company	Register/Login, Manage Company Profile, Post Task, View Ranked Candidates, Review Application, Update Application Status, Schedule Interview, Provide Feedback
Administrator	Manage Users, Verify Companies, Moderate Tasks, View Analytics, Manage Permissions
AI Matching Service	Compute Match Score, Rank Candidates, Recommend Tasks

4.2 Functional Modeling

Functional modeling describes how data is structured, stored, and transformed as it flows through the system to satisfy the use cases defined earlier. While use case modeling focuses on the interaction between actors and the system, functional modeling focuses on the information itself, capturing both the static structure of the data and the dynamic movement of that data between processes [26]. For the Smart AI Micro-Internship Program, functional modeling is presented through an Entity Relationship Diagram, which defines the persistent data structure, and a set of Data Flow Diagrams, which describe how information moves between users, processes, and data stores. Together these models bridge the gap between the requirements and the database and component design that follow. They also make it possible to verify that every piece of information required by a use case has a clearly defined source, store, and destination within the system.

4.2.1 Entity Relationship Diagram

An Entity Relationship Diagram (ERD) is a data model that represents the entities of a system, their attributes, and the relationships between them, providing a clear blueprint for the underlying database [27]. Entities correspond to the real-world objects the system stores information about, relationships express how those objects are associated, and cardinality constraints describe how many instances of one entity may relate to another. For the Smart AI Micro-Internship Program, the data layer is implemented as a relational database, so the ERD maps directly onto the tables and foreign keys used in the system. The principal entities are **User**, **Student**, **Company**, **Admin**, **Task**, **Application**, **Interview**, and several supporting entities such as **StudentSkill**, **TaskSkill**, and **ApplicationStatusHistory**. These entities and their relationships are summarised in Table 4.7 and described in the paragraphs that follow.

At the centre of the model is the **User** entity, which stores the shared account information such as email, hashed password, and role, and is connected through a one-to-one relationship to exactly one role profile: a **Student**, a **Company**, or an **Admin**. This separation keeps authentication concerns independent from role-specific data and allows each role to be extended without affecting the others. A **Student** is associated with

Table 4.5: Use Case Description: Apply for Task

Use Case Name	Apply for Task
Actors	Student (primary), AI Matching Service (supporting)
Description	Allows a registered student to submit an application for an open task.
Preconditions	The student is logged in, the profile is sufficiently complete, and the task is active with available application slots.
Main Flow	1. The student opens a task and selects "Apply". 2. The system displays the application form. 3. The student enters a cover letter and attaches a resume and optional portfolio. 4. The system validates the inputs. 5. The AI Matching Service computes a match score for the application. 6. The system stores the application with status `submitted` and notifies the company.
Alternative Flows	4a. If the cover letter is too short or a required field is missing, the system shows a validation error and the student corrects the input. 3a. If the student has already applied to this task, the system prevents a duplicate application.
Postconditions	A new application is recorded, linked to the task and student, with an initial status and a computed match score.

several child entities, including skills, education, experience, projects, and certificates, which together make up the rich profile used by the matching engine. A **Company** owns many **Task** entities, and each task in turn references the skills it requires, giving the system the structured data needed for accurate matching.

The transactional core of the model is formed by the **Application** entity, which connects a **Student** to a **Task** in a many-to-one relationship on both sides, while a unique constraint prevents a student from applying to the same task twice. Each application carries a status drawn from a fixed lifecycle and stores a computed match score produced by the AI Matching Service. When an application is shortlisted, an **Interview** entity may be created and linked to the application, task, student, and company, capturing the scheduling details and outcome of the meeting. Every change to an application's status is recorded in the **ApplicationStatusHistory** entity, which provides a complete audit trail and supports transparency for both students and companies.

The entity descriptions and relationships above completely specify the structure that the ERD must represent, and they map one-to-one onto the relational schema detailed later in the physical design chapter. By defining entities, attributes, keys, and cardinalities at this stage, the model guarantees referential integrity and removes ambiguity before any tables are created. This data structure forms the foundation on which the data flow diagrams in the next subsection operate.

Table 4.6: Use Case Description: Schedule Interview

Use Case Name	Schedule Interview
Actors	Company (primary), Student (secondary)
Description	Allows a company to schedule an interview for a shortlisted application.
Preconditions	The company is logged in and owns a task with at least one shortlisted application.
Main Flow	1. The company opens the shortlisted application. 2. The company selects "Schedule Interview". 3. The system displays the interview form. 4. The company sets the date, time, duration, mode, and meeting details. 5. The system validates the schedule and stores the interview. 6. The application status changes to `interview_scheduled` and both parties are notified.
Alternative Flows	5a. If the selected time is in the past or invalid, the system rejects the request and asks for a correction. 4a. If the company later reschedules, the system enforces a maximum of three reschedules per interview.
Postconditions	An interview record is created and linked to the application, task, student, and company.

4.2.2 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical model that shows how data moves through a system, from external sources, through processes that transform it, into data stores, and out to external destinations [26]. A DFD is built from four basic elements: external entities, processes, data stores, and data flows, and it deliberately omits control logic and timing so that attention stays on the information itself. DFDs are typically organised into levels, where each level adds more detail to the processes shown at the level above, a technique known as levelled decomposition. For the Smart AI Micro-Internship Program, three levels of DFD are presented: a context diagram (Level 0) that shows the system as a whole, a Level 1 diagram that decomposes the system into its major processes, and a Level 2 diagram that expands the most important process in further detail. This progressive refinement keeps each diagram readable while still capturing the complete data flow of the platform.

4.2.2.1 DFD Level 0 The Level 0 diagram, also called the context diagram, represents the entire Smart AI Micro-Internship Program as a single process and shows only its interactions with the external entities that surround it. The external entities are the **Student**, the **Company**, and the **Administrator**, each of which exchanges high-level data flows with the system. A student sends registration details, profile information, and applications into the system and receives task recommendations, application updates, and interview notifications in return. A company sends company details, task postings, and hiring decisions into the system and receives applications and ranked candidate lists, while the administrator sends moderation and verification actions and receives platform ana-

Table 4.7: Principal Entities and Relationships

Entity	Key Relationships	Description
User	1:1 with Student / Company / Admin	Stores authentication and account data common to all roles.
Student	1:M with Application, Interview; 1:M with Skill, Education, Experience, Project	Holds the academic and professional profile of a student.
Company	1:M with Task, Interview	Holds the verified profile of an employer organisation.
Task	1:M with Application; M:1 with Company	Represents a micro-internship opportunity posted by a company.
Application	M:1 with Task and Student; 1:1 with Interview	Records a student's application to a task and its status.
Interview	M:1 with Application, Task, Student, Company	Represents a scheduled interview between a student and a company.
StudentSkill / TaskSkill	M:1 with Student / Task	Stores the individual skills owned by a student or required by a task.
ApplicationStatusHistory	M:1 with Application	Audit trail of every status change of an application.

lytics. This single-process view defines the overall boundary of the system and confirms exactly which information enters and leaves it before any internal detail is considered.

4.2.2.2 DFD Level 1 The Level 1 diagram decomposes the single process of the context diagram into the major functional processes of the platform, namely **Manage Accounts**, **Manage Profiles**, **Manage Tasks**, **Process Applications**, **AI Matching**, **Manage Interviews**, and **Administration**. Each process reads from and writes to one or more data stores, which correspond to the main entities of the ERD such as Users, Students, Companies, Tasks, Applications, and Interviews. For example, the **Process Applications** process receives an application from a student, stores it in the Applications data store, and requests a score from the **AI Matching** process, which reads the relevant student and task data to compute the result. The **Manage Interviews** process reads accepted or shortlisted applications and writes interview records that are then sent as notifications to both the student and the company. This level makes the internal structure of the system explicit and shows how the data stores connect the otherwise independent processes.

4.2.2.3 DFD Level 2 The Level 2 diagram expands the most significant process, **AI Matching**, into its internal sub-processes to show exactly how a match score is produced. The first sub-process, **Collect Inputs**, gathers the student's skills, experience, interests,

and profile text together with the task's required skills, category, and description. The second sub-process, **Compute Component Scores**, calculates separate scores for skill match, experience fit, level fit, interest overlap, and textual similarity, the last of which is obtained using a text-similarity technique. The third sub-process, **Combine and Rank**, applies the configured weights to these component scores to produce a single overall score between zero and one hundred, and then ranks tasks for a student or candidates for a task accordingly. The final result is written back to the Applications data store and returned to the calling process, while the component breakdown and human-readable reasons are also recorded so that the result is transparent and explainable.

The three levels of data flow diagram described above provide a complete picture of how information is created, transformed, and consumed across the Smart AI Micro-Internship Program. The textual specifications given here define precisely the external entities, processes, data stores, and flows that each diagram must contain, ensuring consistency with both the use case model and the entity relationship diagram. Having established what the system does and how its data flows, the report now proceeds to the structural and behavioural design that determines how these functions are realised.

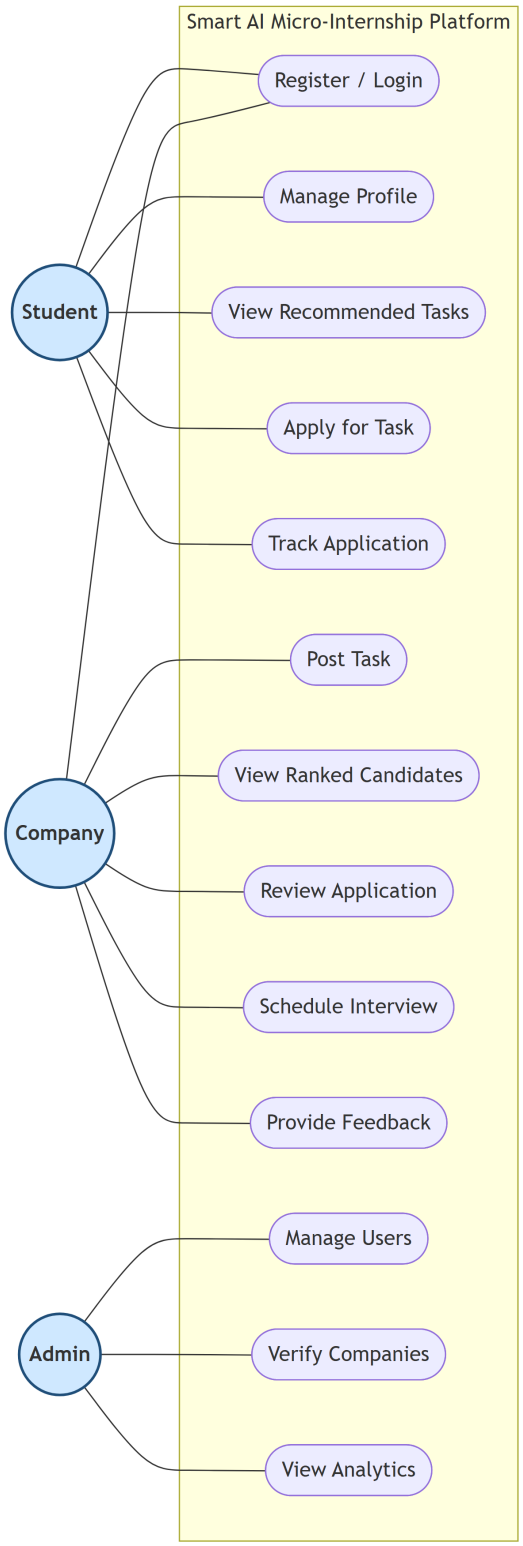

Figure 4.1: Use Case Diagram of the Smart AI Micro-Internship Program

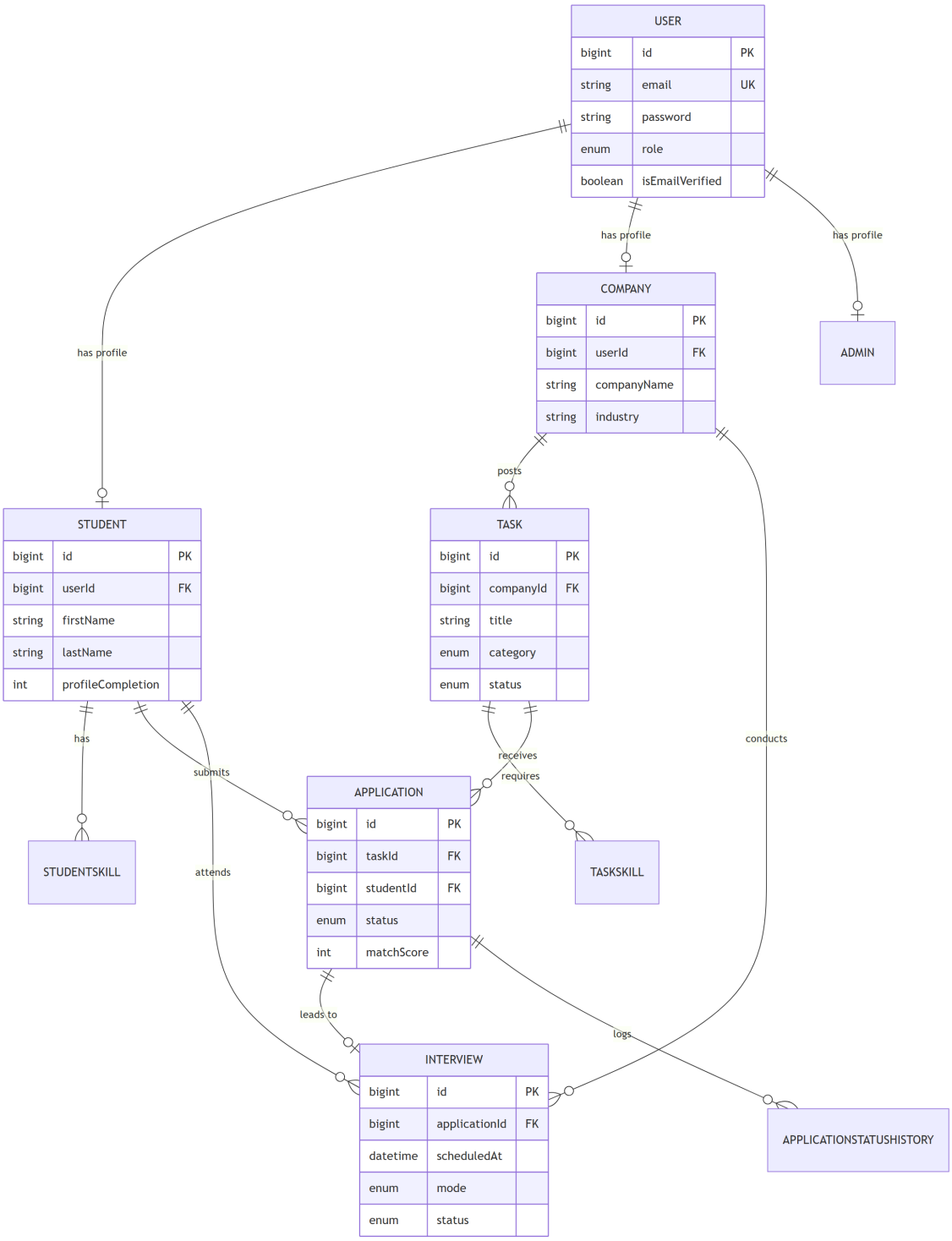

Figure 4.2: Entity Relationship Diagram of the Smart AI Micro-Internship Program

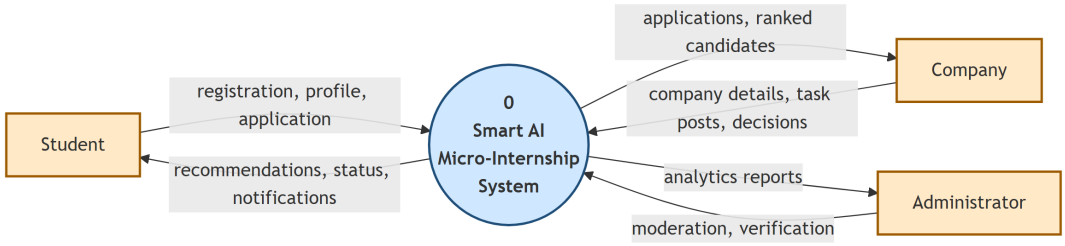

Figure 4.3: Data Flow Diagram — Level 0 (Context Diagram)

Figure 4.4: Data Flow Diagram — Level 1 (Major Processes)

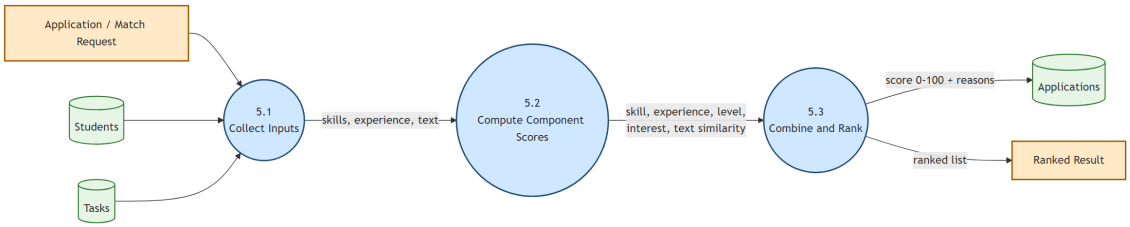

Figure 4.5: Data Flow Diagram — Level 2 (AI Matching Process)

Chapter 5

SYSTEM DESIGN

With the functional analysis from the previous chapter complete, this chapter answers the internal engineering question: given what the platform must do, how should it be organised and built? Our design decisions are expressed using UML diagrams, split into two complementary perspectives. The structural perspective captures the static architecture — the key domain classes and their relationships, and the physical infrastructure on which the software runs. The behavioural perspective shows the system in motion — how control flows through a business process, how collaborating objects message each other, and how a single user action unfolds through the system step by step. Architecturally, the Smart AI Micro-Internship Program follows a layered, service-oriented structure: a Next.js front-end communicates over HTTPS with a Node.js/Express back-end, which persists data in a relational database and offloads intelligent matching work to a dedicated Python AI service that runs as a separate deployable unit.

5.1 Structure Diagrams

The two structure diagrams in this section describe the platform's architecture from a static standpoint — independently of when or in what order anything happens at runtime [28]. The class diagram works at the logical level, showing the domain objects around which the code is organised and the associations, dependencies, and generalisations between them. The deployment diagram works at the operational level, showing which servers and network boundaries the running application spans. Together they give a complete structural picture: the class diagram guides implementation decisions, while the deployment diagram supports infrastructure planning and helps reason about where security boundaries and scaling bottlenecks lie.

5.1.1 Class Diagram

A class diagram is a structural UML diagram that describes the classes in a system, their attributes and methods, and the associations, generalisations, and dependencies between them [29]. It serves as a blueprint for the object-oriented implementation of the system and maps closely to the entities defined in the data model. For the Smart AI Micro-Internship Program, the principal classes are `User`, `Student`, `Company`, `Admin`, `Task`, `Application`, and `Interview`, together with supporting classes such as `StudentSkill` and `TaskSkill`. The `User` class encapsulates authentication behaviour, exposing methods such as `comparePassword()`, `generateAuthToken()`, and `generateOTP()`, while role-specific classes extend it with their own attributes and operations. The `Task` class, for example, provides domain methods such as `canApply()`, `getMatchingScore()`, and `getTimeRemaining()`, which encapsulate the business rules that govern task availability.

The associations between these classes reflect the relationships described in the entity model. A `User` is linked to exactly one role class through a one-to-one association, while a `Company` is linked to many `Task` objects and a `Student` is linked to many `Application`

objects through one-to-many associations. The `Application` class acts as an association class connecting `Student` and `Task`, and it may be linked to a single `Interview` object when a candidate is shortlisted. Encapsulating both data and behaviour within these classes keeps the design cohesive and supports maintainability, since each class is responsible for enforcing its own invariants. The detailed attributes and data types of each class are listed in the physical design chapter, which complements this logical view with the concrete database structure.

5.1.2 Deployment Diagrams

A deployment diagram is a structural UML diagram that shows the physical arrangement of a system, mapping software artifacts onto the hardware nodes and execution environments on which they run [28]. It models nodes, such as servers and client devices, and the communication paths between them, making the runtime topology of the system explicit. For the Smart AI Micro-Internship Program, the deployment model is composed of four main nodes: the **Client** node, which runs the front-end application in a web browser; the **Web/Application Server** node, which hosts the Next.js front-end and the Node.js/Express back-end; the **AI Service** node, which runs the Python FastAPI matching service; and the **Database Server** node, which hosts the relational database. The client communicates with the application server over HTTPS, the back-end communicates with the AI service over an internal HTTP interface, and the back-end communicates with the database through a secure database connection.

This separation of nodes provides several design benefits that align with the non-functional requirements identified earlier. Isolating the AI service on its own node allows the computationally heavier matching workload to be scaled independently of the main web traffic, improving overall performance and scalability. Keeping the database on a dedicated node enhances security and reliability, since access can be restricted to the application server and backups can be managed centrally. The stateless nature of the application and AI tiers means that additional instances of either can be added behind a load balancer to handle increased demand, supporting the future growth of the platform. This deployment structure therefore translates the architectural goals of the system into a concrete and operationally robust configuration.

5.2 Behavioral Diagrams

While structure diagrams show the platform's skeleton, behavioural diagrams show it in motion [29]. Three types of behavioural diagram are used here, each chosen because it answers a different question about the system's runtime behaviour. Activity diagrams answer "what is the sequence of steps in this business process?" — they are especially useful for the task-application workflow, which involves decision branches for eligibility, validation, and AI scoring. Communication diagrams answer "which objects collaborate and through what named links?" — they help verify that the design is loosely coupled and that no single class carries too much responsibility. Sequence diagrams answer "in what exact order do messages flow for this specific scenario?" — they are essential for designing the authentication and data-retrieval interactions correctly before a single line of code is written. The subsections below present one representative diagram of each type.

5.2.1 Activity Diagrams

An activity diagram is a behavioural UML diagram that models the flow of control from one activity to another within a process, including decisions, parallel branches, and synchronisation points [28]. It is particularly useful for visualising business workflows that span several steps and involve conditional logic. For the Smart AI Micro-Internship Program, a representative activity diagram models the end-to-end application workflow, beginning when a student opens a task and ending when a hiring decision is recorded. The flow starts with the student viewing a task, then branches on whether the student is eligible to apply; if eligible, the student completes and submits the application form, the system validates the inputs, and the AI Matching Service computes a match score. The application then enters the company's review process, where decision nodes route it toward shortlisting, interviewing, acceptance, or rejection, with each outcome updating the application status accordingly.

This workflow demonstrates how the system enforces business rules through guarded transitions and how it handles both the normal path and exceptional cases. For instance, a decision node checks whether the application deadline has passed or the maximum number of applications has been reached, redirecting the student to an appropriate message rather than allowing an invalid submission. Another decision node ensures that an interview can only be scheduled for an application that has been shortlisted, preserving the integrity of the application lifecycle. By modelling these branches explicitly, the activity diagram provides a clear specification for validating every input and transition, which directly supports the goal of robust input handling. The same modelling approach is applied to other key processes, such as task posting and profile completion, to ensure consistent and predictable behaviour across the platform.

5.2.2 Communication Diagrams

A communication diagram, formerly known as a collaboration diagram, is a behavioural UML diagram that focuses on the structural organisation of the objects that participate in an interaction and the messages that pass between them [29]. Unlike a sequence diagram, which arranges messages along a vertical time axis, a communication diagram arranges the participating objects spatially and numbers the messages to indicate their order. For the Smart AI Micro-Internship Program, a communication diagram for the task-recommendation interaction shows the collaboration between the `Student` interface, the `TaskController`, the `MatchingService`, and the `Task` and `Student` data objects. The numbered messages show the student requesting recommendations, the controller retrieving the relevant profile and tasks, the matching service computing scores, and the ranked results being returned to the student.

This view is valuable because it highlights how strongly the various components are coupled and which objects bear the most responsibility within an interaction. By examining the links in the diagram, the design can be checked for unnecessary dependencies, and responsibilities can be redistributed to keep the system loosely coupled and cohesive [23]. In the Smart AI Micro-Internship Program, the communication diagram confirms that the controller acts as a coordinator while the matching logic is delegated entirely to the dedicated service, keeping the responsibilities cleanly separated. This separation makes the system easier to test and to extend, since the matching algorithm can evolve

without affecting the controller or the user interface.

5.2.3 Sequence Diagrams

A sequence diagram is a behavioural UML diagram that shows how objects interact over time by arranging the participating lifelines horizontally and ordering the messages between them along a vertical time axis [28]. It is the most effective diagram for describing the precise order of operations in a single scenario, including the requests sent, the responses returned, and the activations of each object. For the Smart AI Micro-Internship Program, sequence diagrams are provided for several fundamental scenarios that recur throughout the system, namely logging in, logging out, changing a password, and viewing stored data. These scenarios were chosen because they exercise the core authentication and data-access mechanisms on which every other feature depends. The following subsections describe each of these sequences in detail.

5.2.3.1 Login The login sequence begins when the user submits an email and password through the sign-in interface, which sends the credentials to the authentication controller on the back-end. The controller retrieves the corresponding `User` record from the database and invokes the `comparePassword()` method, which compares the supplied password against the stored bcrypt hash. If the credentials are valid and the account is active and verified, the controller calls `generateAuthToken()` to create a signed JSON Web Token, which is returned to the client and stored for use in subsequent requests. If the credentials are invalid, the controller returns an authentication error and the interface prompts the user to try again, ensuring that no token is issued for a failed attempt. This sequence shows how security is enforced at the very first interaction and how the token-based mechanism enables stateless authentication for later requests.

5.2.3.2 Logout The logout sequence begins when the authenticated user selects the logout option in the interface, which triggers a request to terminate the current session. Because the system uses stateless JSON Web Tokens, the primary action is the removal of the stored token from the client, after which the user is redirected to the public landing or sign-in page. The interface clears any cached user data so that protected views are no longer accessible without re-authentication, and the back-end optionally records the event for auditing. Any subsequent request that lacks a valid token is rejected by the authentication middleware, guaranteeing that protected resources cannot be reached after logout. This sequence demonstrates how the system reliably ends a session and protects user data once the user has left the platform.

5.2.3.3 Change Password The change-password sequence begins when an authenticated user opens the account settings and submits the current password together with a new password. The controller first verifies the current password using `comparePassword()` to confirm the user's identity before any change is made. If verification succeeds, the new password is assigned to the `User` record, and a model hook automatically hashes it with bcrypt before it is persisted, so that the plaintext password is never stored. The system then confirms the successful update to the user and, depending on configuration, may invalidate existing sessions so that the new credentials must be used. If the current password is incorrect, the request is rejected and the user is asked to re-enter it, preventing unauthorised password changes.

5.2.3.4 View Database The view-data sequence describes how an authorised user retrieves stored records, such as a company viewing the applications submitted for one of its tasks. The interface sends a request that includes the user's authentication token, which the authentication middleware validates before the request is allowed to proceed. The controller then checks that the user is authorised to access the requested data, for example by confirming that the company owns the task whose applications are being requested. Once authorisation is confirmed, the controller queries the database, retrieves the matching records along with their related data, and returns them to the interface for display. This sequence shows how every data-access operation is protected by both authentication and authorisation checks, ensuring that users can only view information they are permitted to see.

The behavioural diagrams described in this section, taken together with the structural diagrams, provide a complete design specification for the Smart AI Micro-Internship Program. They define both the static organisation of the system and the dynamic interactions that realise its functionality, giving a clear and verifiable basis for implementation. The next chapter builds on this design by presenting the system's user interfaces and the physical structure of the database tables that store its data.

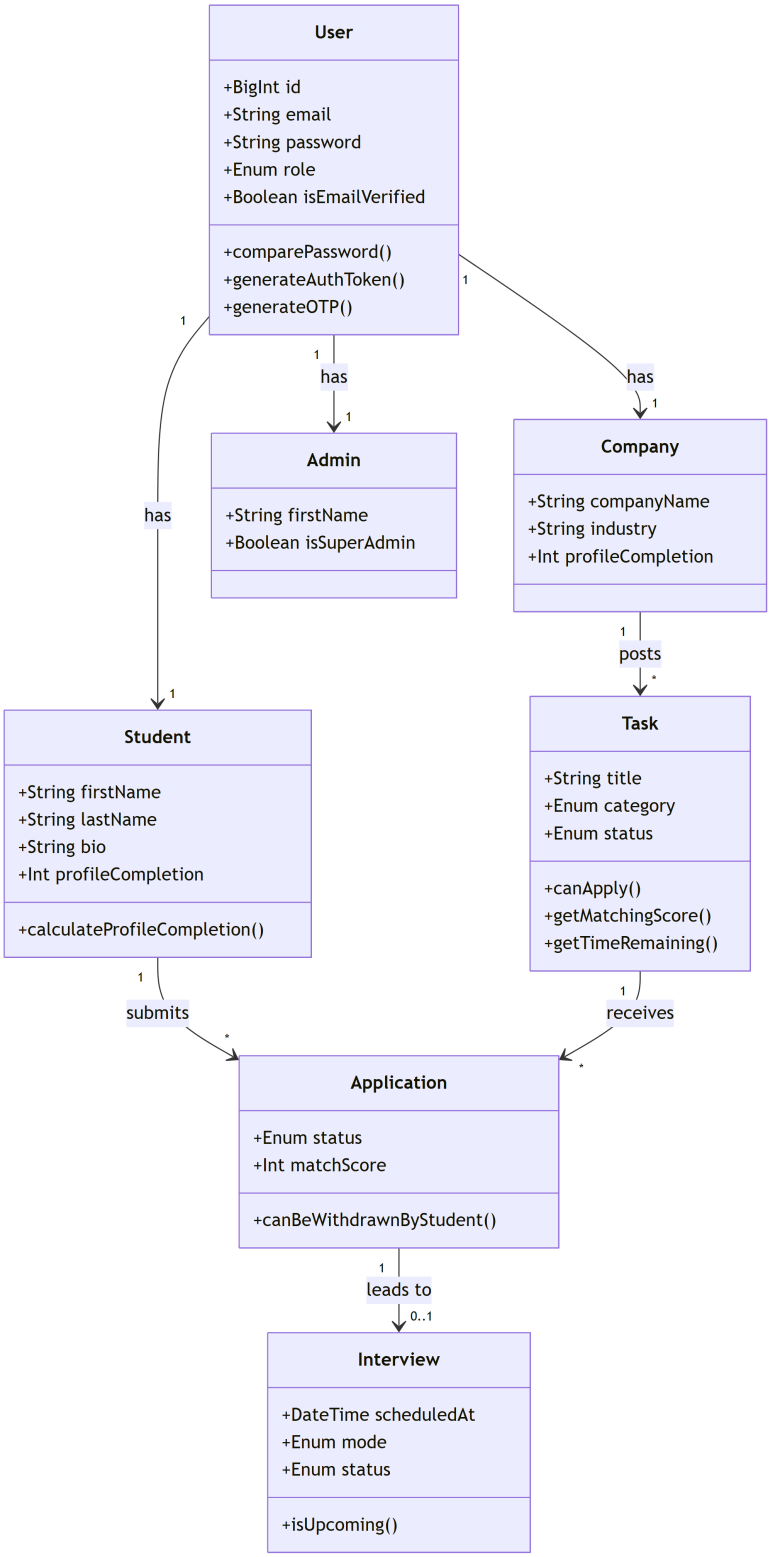

Figure 5.1: Class Diagram of the Smart AI Micro-Internship Program

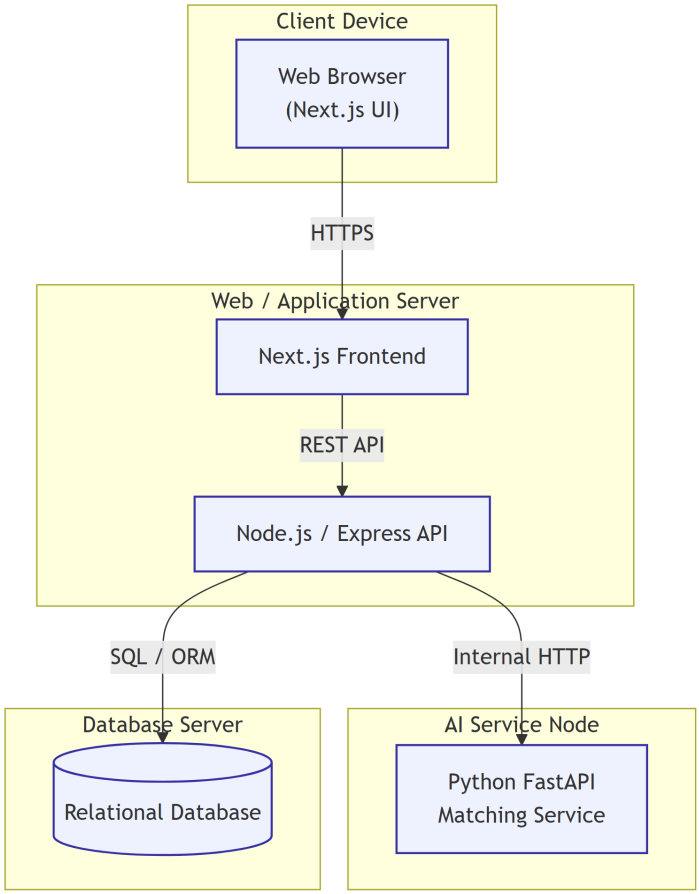

Figure 5.2: Deployment Diagram of the Smart AI Micro-Internship Program

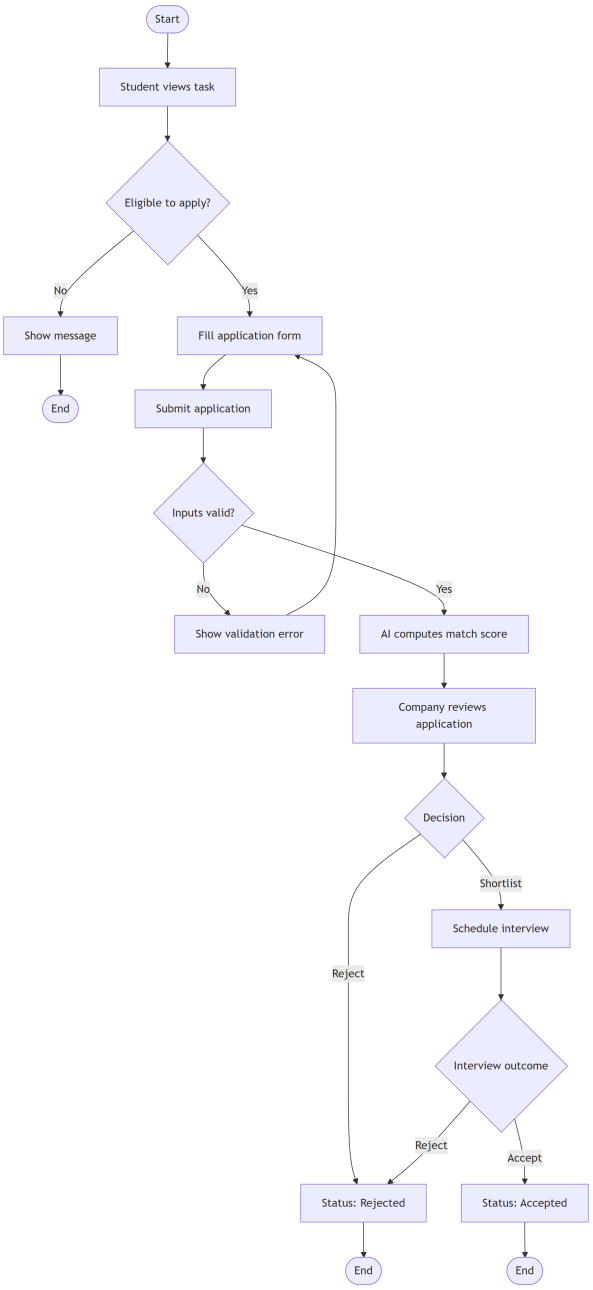

Figure 5.3: Activity Diagram for the Task Application Workflow

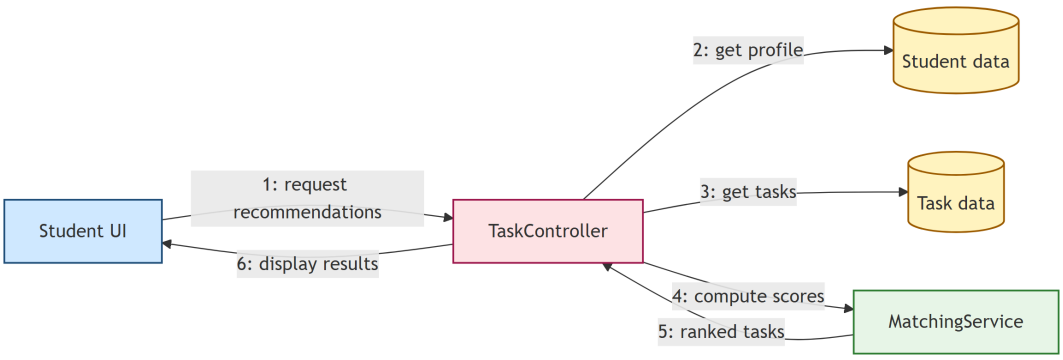

Figure 5.4: Communication Diagram for the Task Recommendation Interaction

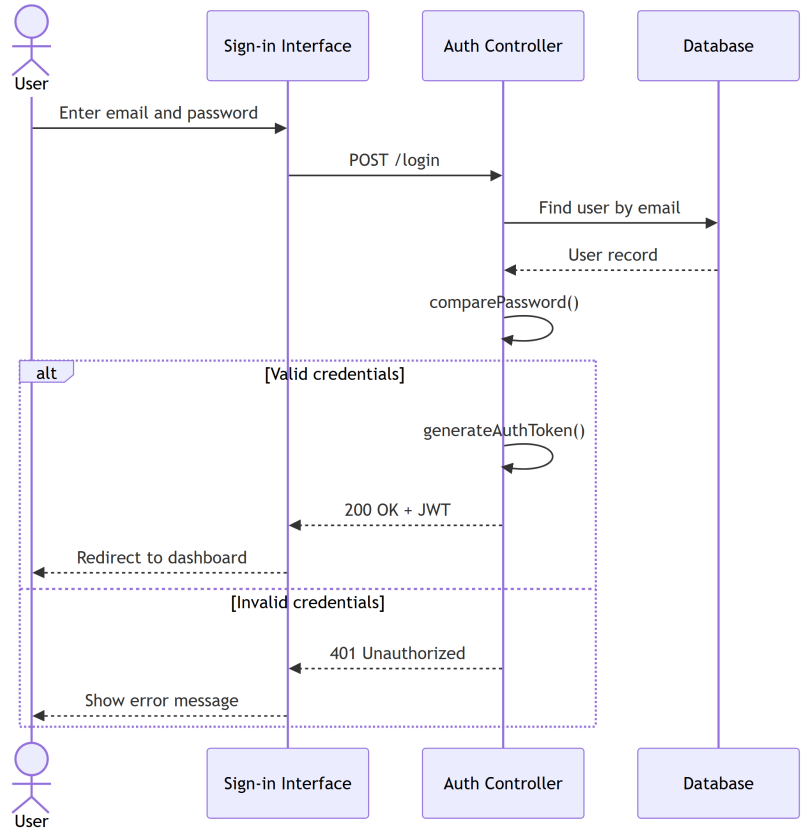

Figure 5.5: Sequence Diagram — Login

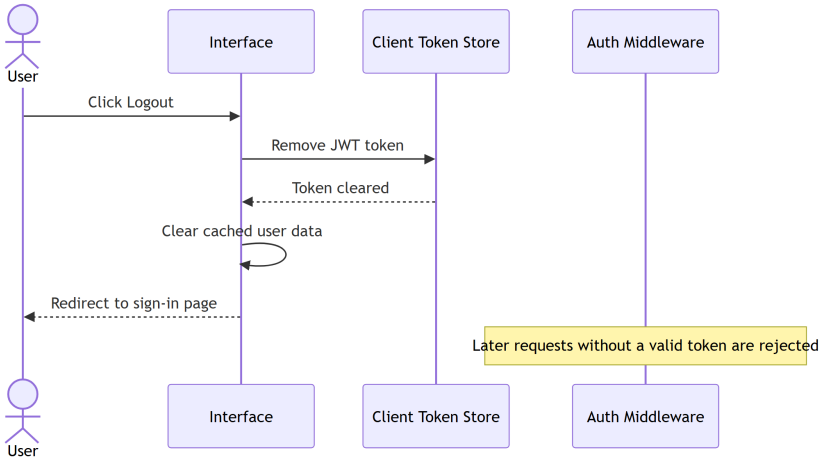

Figure 5.6: Sequence Diagram — Logout

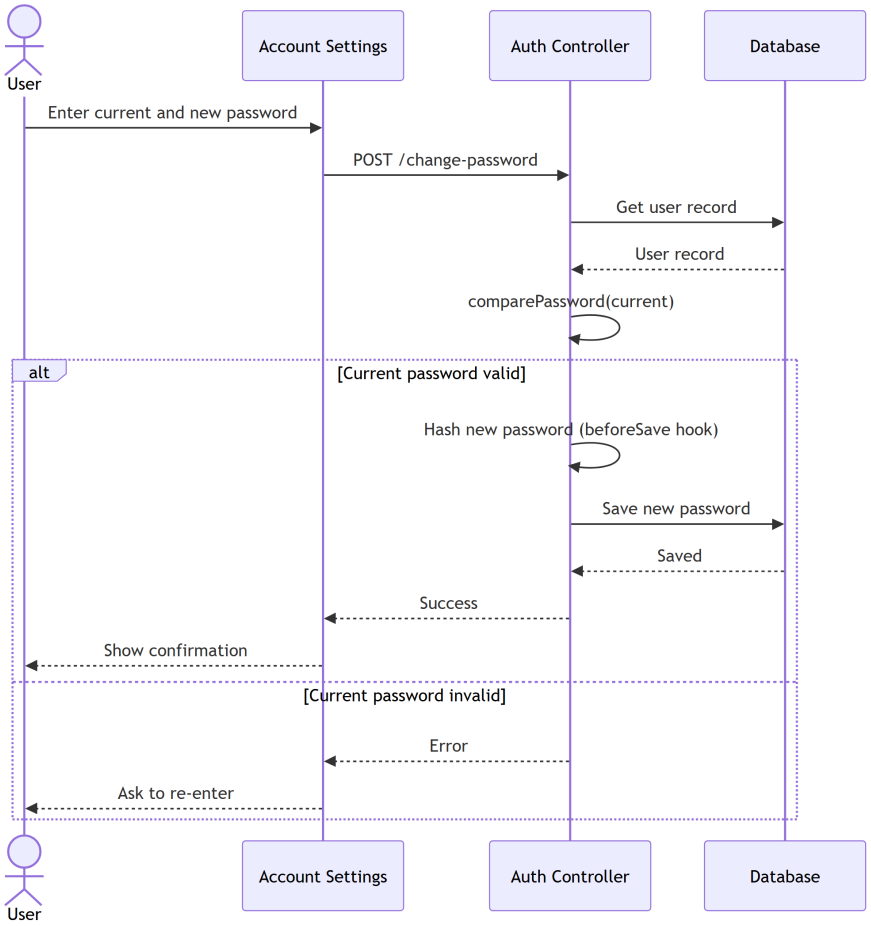

Figure 5.7: Sequence Diagram — Change Password

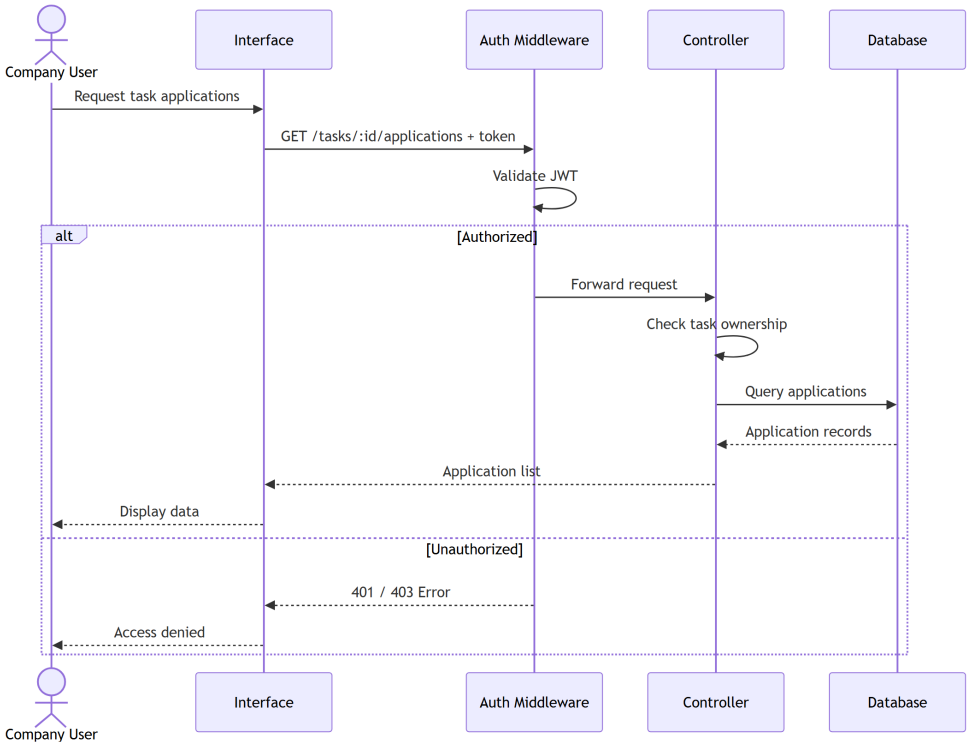

Figure 5.8: Sequence Diagram — View Stored Data

Chapter 6

SYSTEM INTERFACE AND PHYSICAL DESIGN

6.1 System User Interfaces

6.1.1 Sign-up Interface

6.1.2 Sign-in Interface

6.1.3 Sign-out Interface

6.1.4 Forget Password Interface

6.1.5 Main Menu Interface

6.2 User Table

6.3 User Log Table

6.4 Admin Table

6.5 Extra Table 1

6.6 Extra Table 2

Chapter 7

TEST PLAN

7.1 Objective of the Testing Phase

7.2 Levels of Tests for Testing Software

7.2.1 Unit Testing

7.2.2 Integration testing

7.2.3 System Testing

7.3 Test Management Process

7.3.1 Design the Test Strategy

7.3.2 Test Objectives

7.3.3 Test Criteria

7.3.4 Resource Planning

7.3.5 Plan Test Environment

7.3.6 Schedule and Estimation

7.4 Test Cases

7.4.1 Test Case 1

7.4.1.1 Test Method

7.4.1.2 Test Case Description

7.4.1.3 Test Case Data

7.4.1.4 Test Case Report

7.5 Bugs Report

Chapter 8

User Manual

8.1 Description

8.2 System Usage Steps (with screenshots)

Chapter 9

REPORTS

9.1 Introduction

9.2 Total Reports

9.3 Area Wise Report

9.4 Report 1

9.5 Report 2

Chapter 10

CONCLUSION

10.1 Conclusion

10.2 Future Work

References

[1] World Economic Forum, "The future of jobs report," 2023, online; Available: https://www.weforum.org.

[2] A. Kumar and S. Gupta, "Machine learning-based job recommendation and candidate ranking systems," *IEEE Transactions on Computational Social Systems*, vol. 8, no. 4, pp. 890–901, 2021.

[3] Coursera Inc., "Coursera online learning platform," 2024, online; Available: https://www.coursera.org.

[4] GitHub, "Github classroom," 2024, online; Available: https://classroom.github.com.

[5] J. B. Lee and M. Chen, "Artificial intelligence in recruitment: Resume parsing and automated screening," *IEEE Access*, vol. 9, pp. 123 456–123 468, 2021.

[6] Upwork Inc., "Upwork: The world's work marketplace," 2024, online; Available: https://www.upwork.com.

[7] Internshala, "Internshala internship and training platform," 2024, online; Available: https://internshala.com.

[8] LinkedIn Corporation, "Linkedin jobs and professional networking platform," 2024, online; Available: https://www.linkedin.com.

[9] Fiverr International Ltd., "Fiverr freelance marketplace," 2024, online; Available: https://www.fiverr.com.

[10] Handshake, "Handshake student career platform," 2024, online; Available: https://www.joinhandshake.com.

[11] Society for Human Resource Management, "Human capital benchmarking report," 2022, online; Available: https://www.shrm.org.

[12] International Labour Organization, "Global employment trends for youth," 2023, online; Available: https://www.ilo.org.

[13] UNESCO, "Global education monitoring report," 2023, online; Available: https://www.unesco.org.

[14] World Bank, "Small and medium enterprises (smes) finance," 2023, online; Available: https://www.worldbank.org.

[15] IEEE, "Agile methodology and incremental development practices," 2020, online; Available: https://www.ieee.org.

[16] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[17] B. W. Boehm, "A spiral model of software development and enhancement," *IEEE Computer*, vol. 21, no. 5, pp. 61–72, 1988.

[18] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill Education, 2019.

[19] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.

[20] G. Kotonya and I. Sommerville, *Requirements Engineering: Processes and Techniques*. John Wiley & Sons, 1998.

[21] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Project Management Institute, 2021.

[22] K. Schwalbe, *Information Technology Project Management*, 9th ed. Cengage Learning, 2019.

[23] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd ed. Prentice Hall, 2004.

[24] A. Cockburn, *Writing Effective Use Cases*. Addison-Wesley, 2000.

[25] M. Cohn, *User Stories Applied: For Agile Software Development*. Addison-Wesley, 2004.

[26] E. Yourdon, *Modern Structured Analysis*. Prentice Hall, 1989.

[27] P. P. Chen, "The entity-relationship model—toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[28] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Addison-Wesley, 2003.

[29] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Addison-Wesley, 2005.